

NETWORK PACKET ANALYSIS

(Wireshark)



Introduction to Wireshark and Its Importance in Cybersecurity

Wireshark is a widely used **network protocol analyzer** that allows security professionals, system administrators, and network engineers to capture and inspect traffic in real time. It provides a **detailed view of network communications**, helping users analyze packets at a granular level.

Why Wireshark is Used in Cybersecurity?

Network Traffic Analysis

1. Helps monitor and analyze real-time data transmission across a network.
2. Identifies unusual traffic patterns that could indicate a cyber attack.

Intrusion Detection and Threat Hunting

1. Detects signs of **malicious activity**, such as unauthorized access or malware communication.
2. Assists in tracking down the **source of an attack** by analyzing traffic behavior.

Packet Inspection and Forensics

1. Examines network packets to identify **malware-infected data** or suspicious payloads.
2. Helps in post-incident investigations to understand how a breach occurred.

Performance Monitoring and Troubleshooting

1. Identifies **network congestion, dropped packets, and misconfigurations** affecting performance.
2. Troubleshoots network latency and connectivity issues efficiently.

Penetration Testing and Ethical Hacking

1. Used to analyze **vulnerabilities in network security protocols**.
2. Helps ethical hackers understand **how attackers can intercept and manipulate network traffic**.

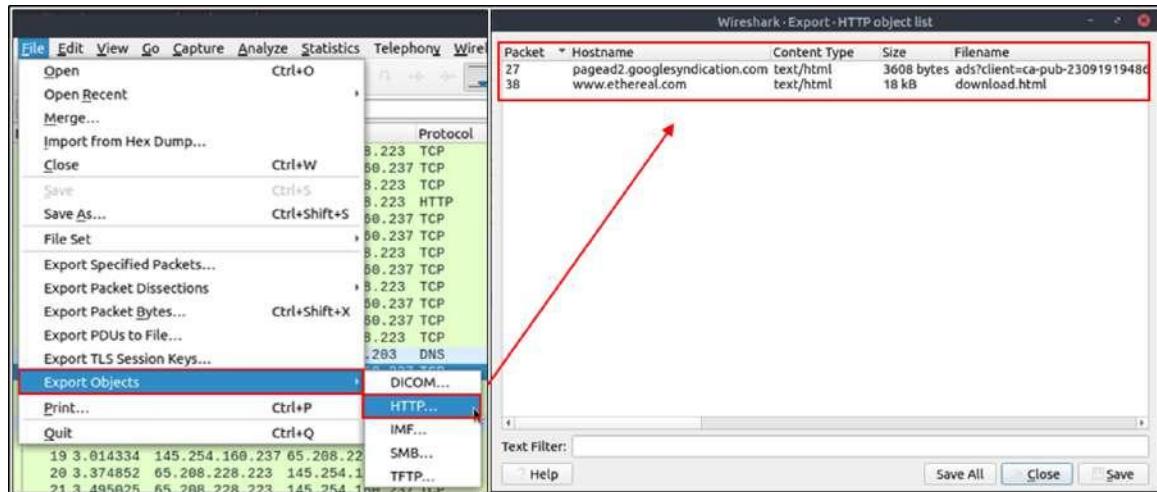
Decrypting Encrypted Traffic (When applicable)

1. Allows inspection of **unencrypted and weakly encrypted data**.
2. Aids in ensuring sensitive information is not exposed over the network.

Wireshark is an essential tool in **cybersecurity defense**, providing visibility into network traffic to identify vulnerabilities, detect attacks, and enhance overall security.

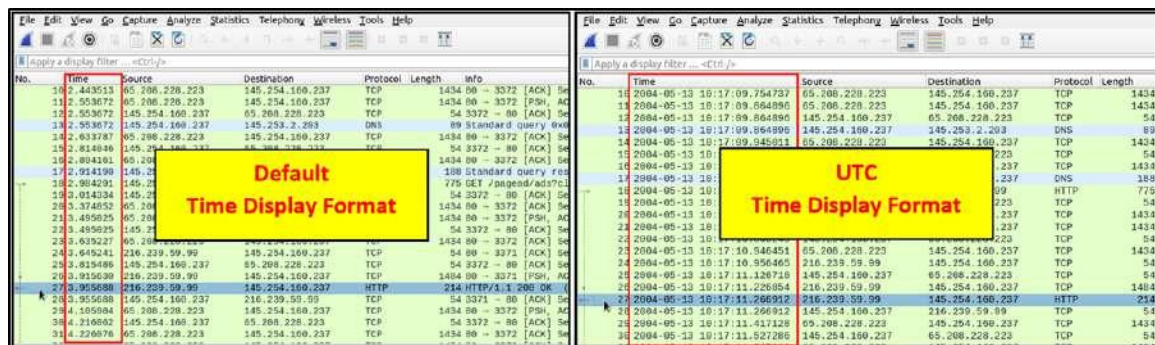
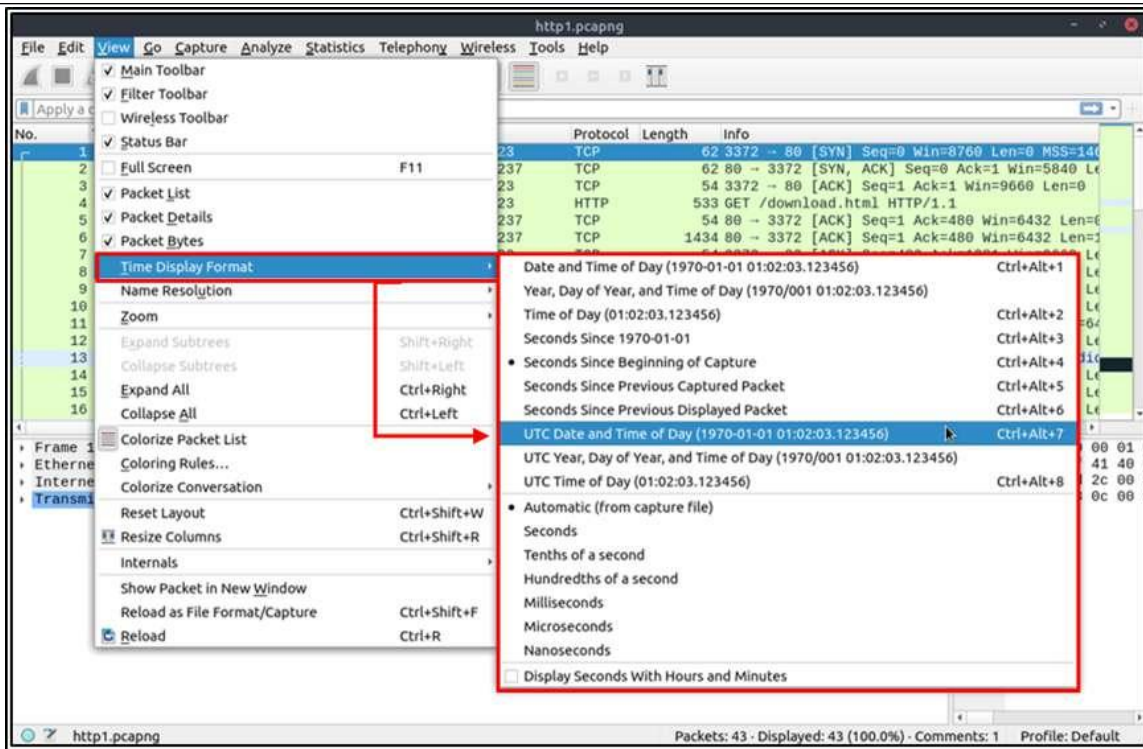
Export Objects (Files)

Wireshark can extract files transferred through the wire. For a security analyst, it is vital to discover shared files and save them for further investigation. Exporting objects are available only for selected protocol's streams (DICOM, HTTP, IMF, SMB and TFTP).



Time Display Format

Wireshark lists the packets as they are captured, so investigating the default flow is not always the best option. By default, Wireshark shows the time in "Seconds Since Beginning of Capture", the common usage is using the UTC Time Display Format for a better view. You can use the "View --> Time Display Format" menu to change the time display format.



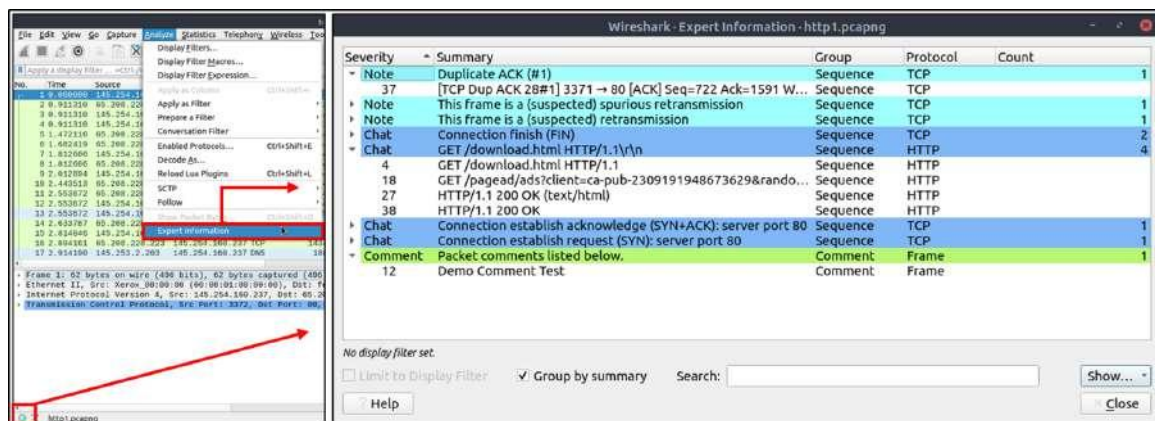
Expert Info

Wireshark also detects specific states of protocols to help analysts easily spot possible anomalies and problems. Note that these are only suggestions, and there is always a chance of having false positives/negatives. Expert info can provide a group of categories in three different severities. Details are shown in the table below.

Severity	Colour	Info
Chat	Blue	Information on usual workflow.
Note	Cyan	Notable events like application error codes.
Warn	Yellow	Warnings like unusual error codes or problem statements.
Error	Red	Problems like malformed packets.

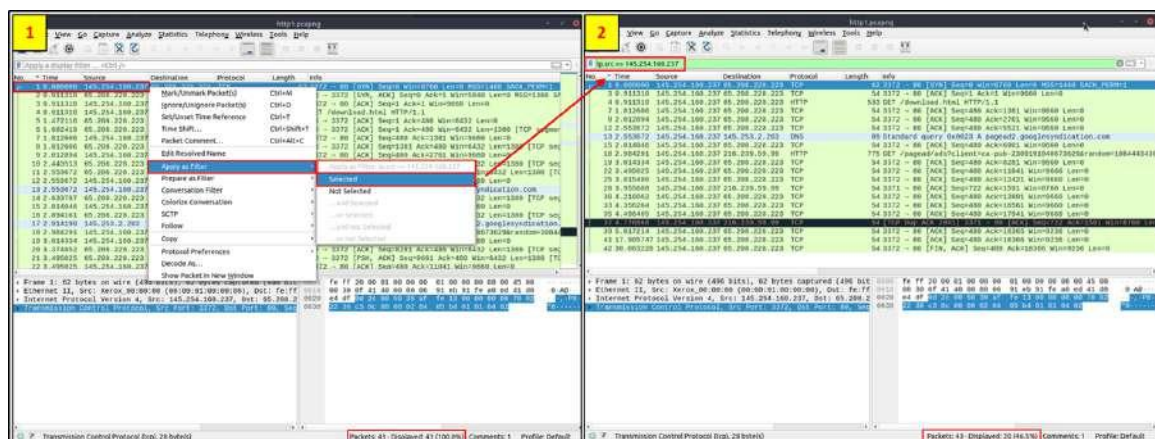
Frequently encountered information groups are listed in the table below. You can refer to Wireshark's official documentation for more information on the expert information entries.

Group	Info	Group	Info
Checksum	Checksum errors.	Deprecated	Deprecated protocol usage.
Comment	Packet comment detection.	Malformed	Malformed packet detection.



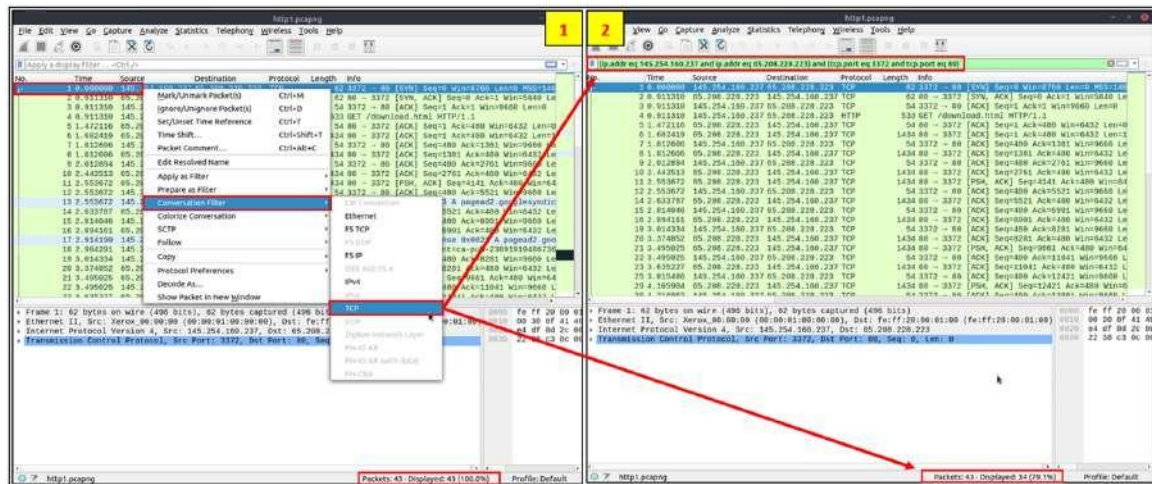
Apply as Filter

This is the most basic way of filtering traffic. While investigating a capture file, you can click on the field you want to filter and use the "right-click menu" or "Analyse --> Apply as Filter" menu to filter the specific value. Once you apply the filter, Wireshark will generate the required filter query, apply it, show the packets according to your choice, and hide the unselected packets from the packet list pane. Note that the number of total and displayed packets are always shown on the status bar.



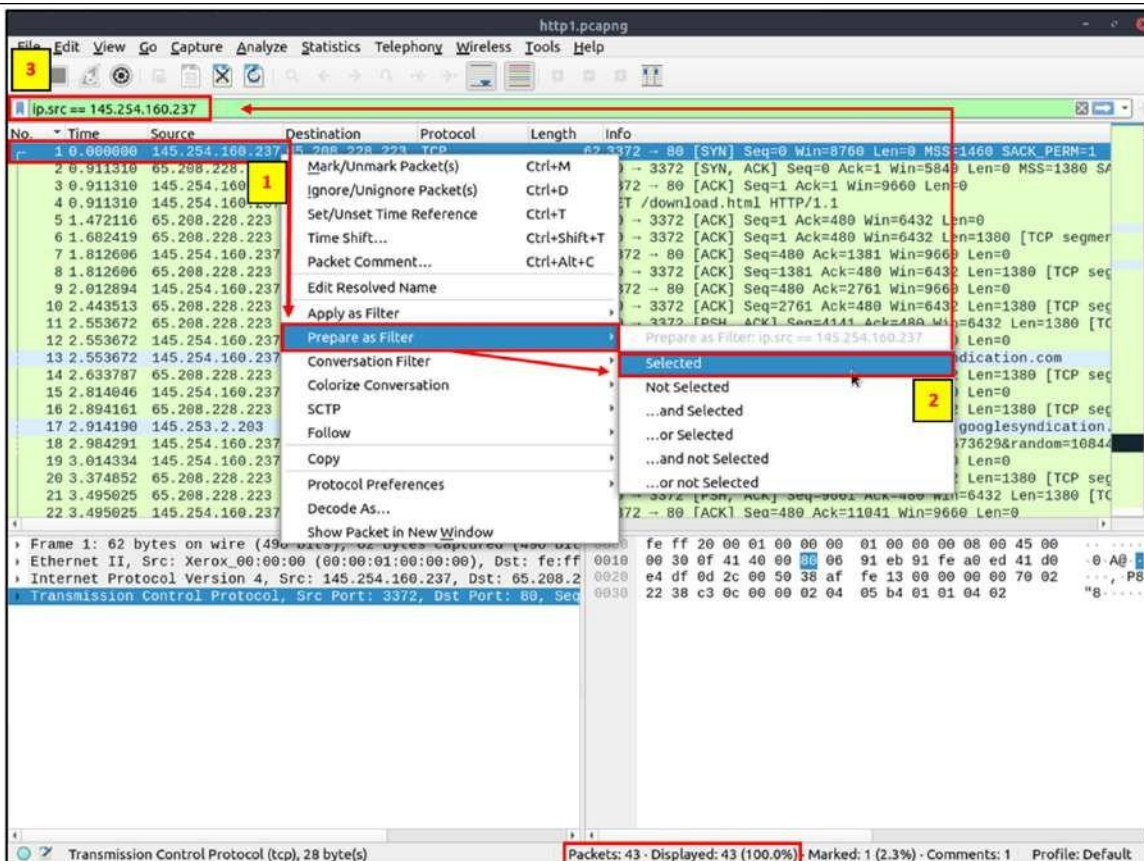
Conversation filter

- "Conversation Filter" option helps you view only the related packets and hide the rest of the packets easily. You can use the "right-click menu" or "**Analyse -> Conversation Filter**" menu to filter conversations.



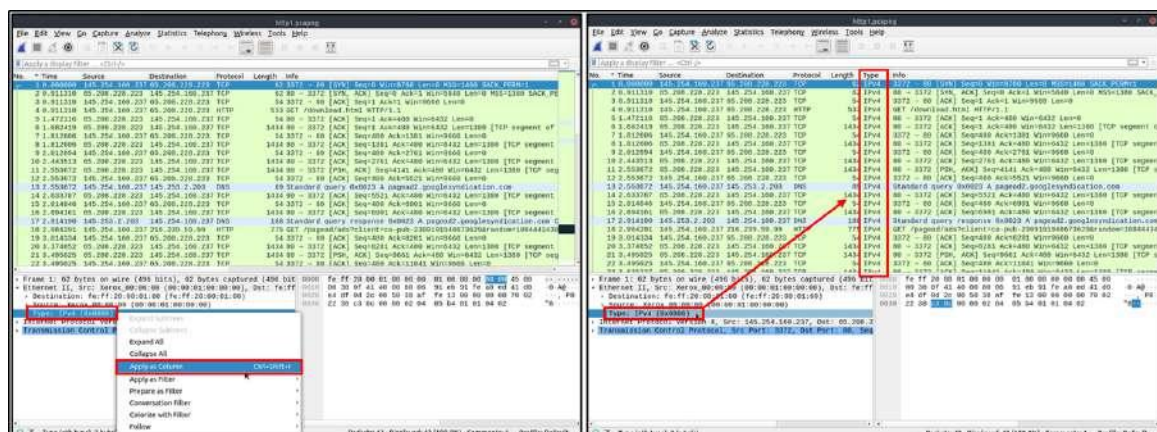
Colourise Conversation

- This option is similar to the "Conversation Filter" with one difference. It highlights the linked packets without applying a display filter and decreasing the number of viewed packets. This option works with the "Colouring Rules" option and changes the packet colours without considering the previously applied colour rule. You can use the "right-click menu" or "**View --> Colourise Conversation**" menu to colourise a linked packet in a single click. Note that you can use the "View --> Colourise Conversation --> Reset Colourisation" menu to undo this operation.



Apply as Column

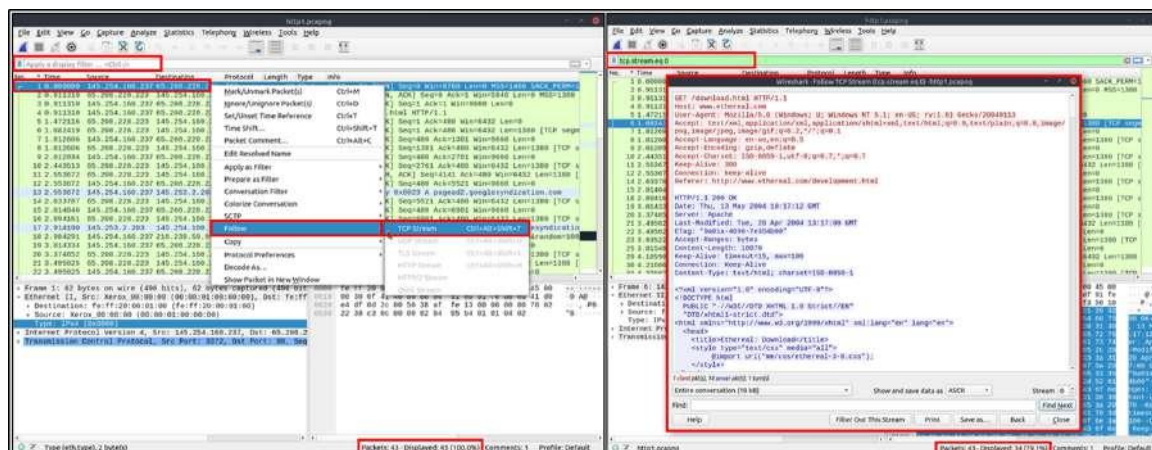
By default, the packet list pane provides basic information about each packet. You can use the "right-click menu" or "Analyse --> Apply as Column" menu to add columns to the packet list pane. Once you click on a value and apply it as a column, it will be visible on the packet list pane. This function helps analysts examine the appearance of a specific value/field across the available packets in the capture file. You can enable/disable the columns shown in the packet list pane by clicking on the top of the packet list pane.



Follow Stream

Wireshark displays everything in packet portion size. However, it is possible to reconstruct the streams and view the raw traffic as it is presented at the application level. Following the protocol, streams help analysts recreate the application-level data and understand the event of interest. It is also possible to view the unencrypted protocol data like usernames, passwords and other transferred data.

You can use the "right-click menu" or "**Analyse --> Follow TCP/UDP/HTTP Stream**" menu to follow traffic streams. Streams are shown in a separate dialogue box; packets originating from the server are highlighted with blue, and those originating from the client are highlighted with red.



Once you follow a stream, Wireshark automatically creates and applies the required filter to view the specific stream. Remember, once a filter is applied, the number of the viewed packets will change. You will need to use the "**X button**" located on the right upper side of the display filter bar to remove the display filter and view all available packets in the capture file.

Statistics

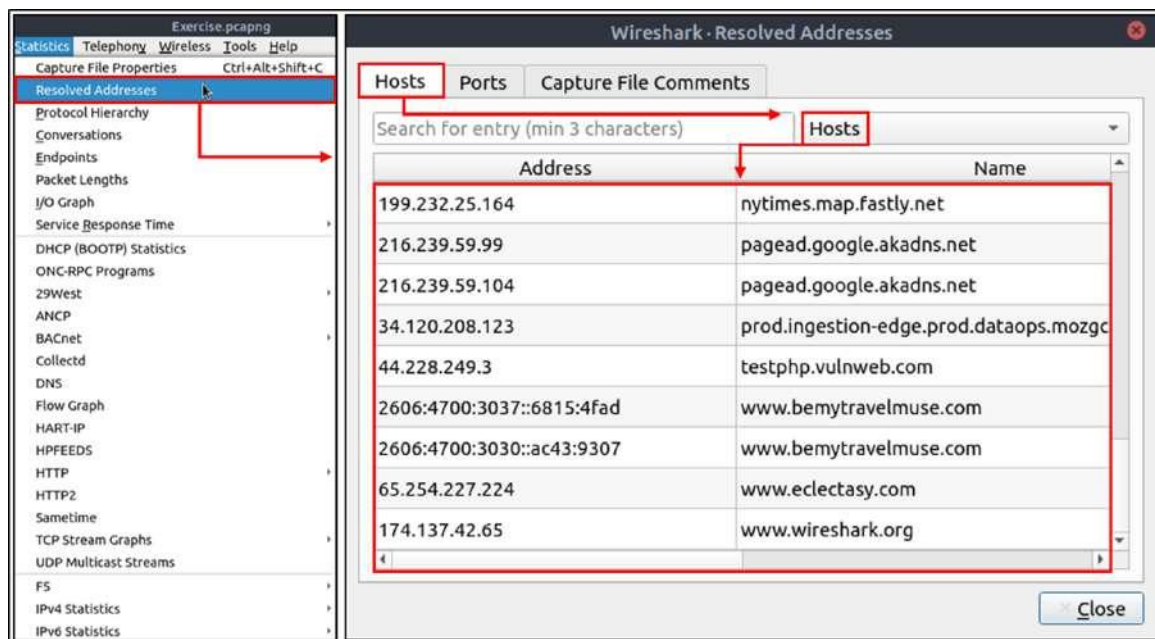
- This menu provides multiple statistics options ready to investigate to help users see the big picture in terms of the scope of the traffic, available protocols, endpoints and conversations, and some protocol-specific details like DHCP, DNS and HTTP/2. For a security analyst, it is crucial to know how to utilise the statistical information. This section provides a quick summary of the processed pcap, which will help analysts create a hypothesis for an investigation. You can use the "**Statistics**" menu to view all available

options. Now start the given VM, open the Wireshark, load the "Exercise.pcapng" file and go through the walkthrough.

Resolved Addresses

This option helps analysts identify IP addresses and DNS names available in the capture file by providing the list of the resolved addresses and their hostnames. Note that the hostname information is taken from DNS answers in the capture file. Analysts can quickly identify the accessed resources by using this menu.

Thus they can spot accessed resources and evaluate them according to the event of interest. You can use the **"Statistics --> Resolved Addresses"** menu to view all resolved addresses by Wireshark.



Protocol Hierarchy

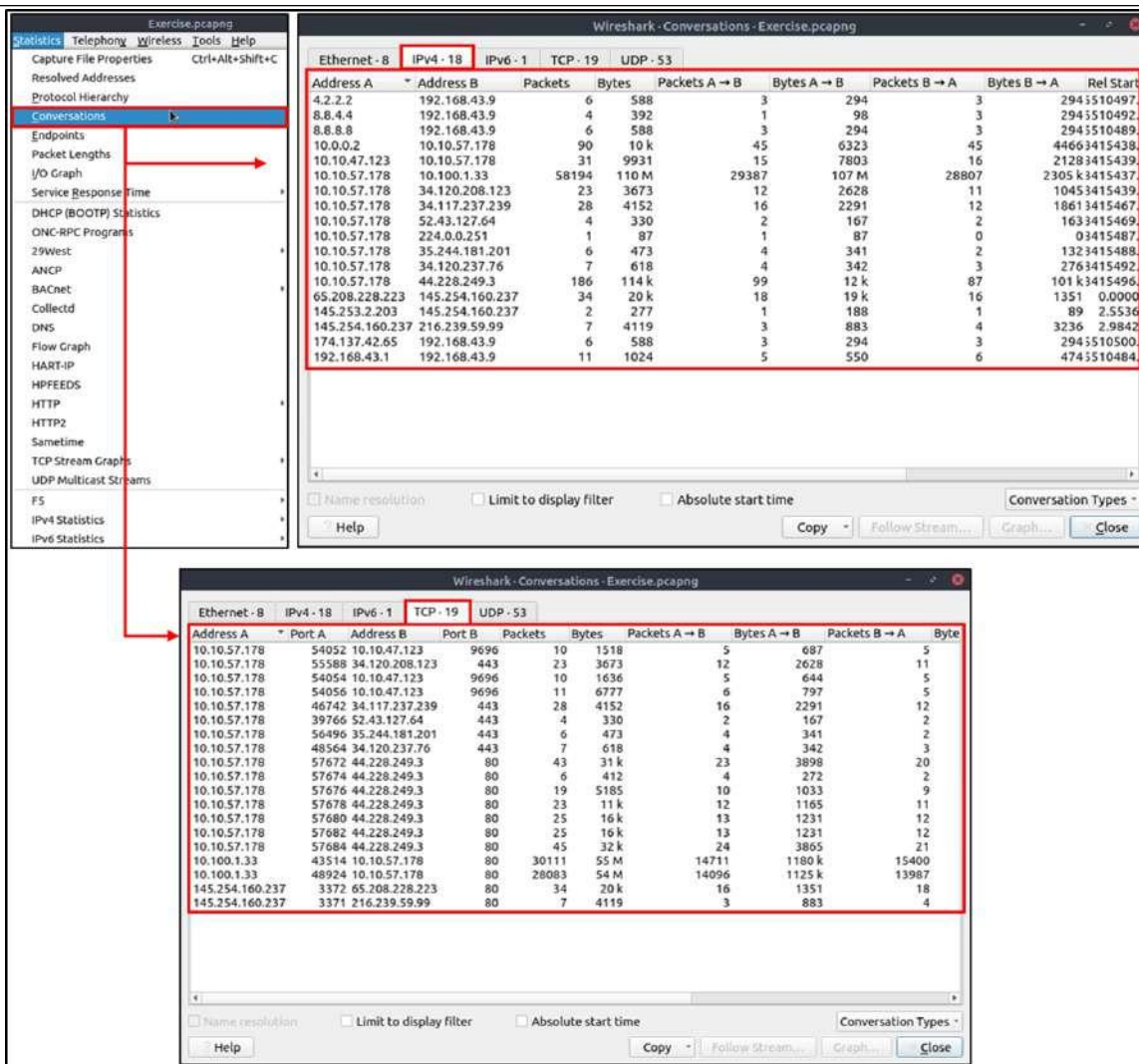
This option breaks down all available protocols from the capture file and helps analysts view the protocols in a tree view based on packet counters and percentages. Thus analysts can view the overall usage of the ports and services and focus on the event of interest. The golden rule mentioned in the previous

room is valid in this section; you can right-click and filter the event of interest. You can use the **"Statistics --> Protocol Hierarchy"** menu to view this info.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes
Frame	100.0	58653	100.0	110243762	1	0	0
Ethernet	100.0	58653	0.7	821142	0	0	0
Internet Protocol Version 6	0.0	1	0.0	40	0	0	0
User Datagram Protocol	0.0	1	0.0	8	0	0	0
Multicast Domain Name System	0.0	1	0.0	45	0	1	45
Internet Protocol Version 4	100.0	58646	1.1	1172920	0	0	0
User Datagram Protocol	0.2	104	0.0	832	0	0	0
Multicast Domain Name System	0.0	1	0.0	45	0	1	45
Domain Name System	0.2	103	0.0	7764	0	103	7764
Transmission Control Protocol	99.8	58520	98.2	108239390	1	57395	101556463
Transport Layer Security	0.1	36	0.0	4742	0	36	4742
Hypertext Transfer Protocol	1.9	1089	6.1	6751343	0	1053	6604075
extensible Markup Language	0.0	1	0.0	18070	0	1	18364
Malformed Packet	0.0	13	0.0	0	0	13	0
Line-based text data	0.0	12	0.0	48307	0	12	38661
JPEG File Interchange Format	0.0	9	0.1	92630	0	7	77826
Malformed Packet	0.0	2	0.0	0	0	2	0
HTML Form URL Encoded	0.0	1	0.0	82	0	1	82
Internet Control Message Protocol	0.0	22	0.0	1408	0	22	1408
Address Resolution Protocol	0.0	6	0.0	168	0	6	168

Conversations

Conversation represents traffic between two specific endpoints. This option provides the list of the conversations in five base formats; ethernet, IPv4, IPv6, TCP and UDP. Thus analysts can identify all conversations and contact endpoints for the event of interest. You can use the "Statistic --> Conversations" menu to view this info.

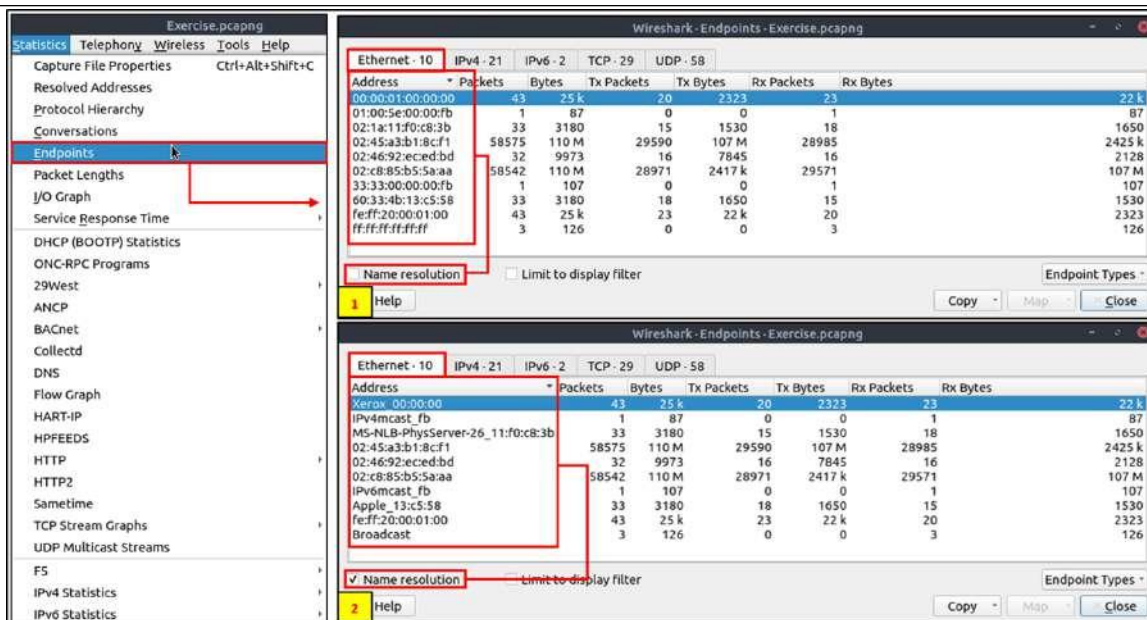


Endpoints

The endpoints option is similar to the conversations option. The only difference is that this option provides unique information for a single information field (Ethernet, IPv4, IPv6, TCP and UDP). Thus analysts can identify the unique endpoints in the capture file and use it for the event of interest. You can use

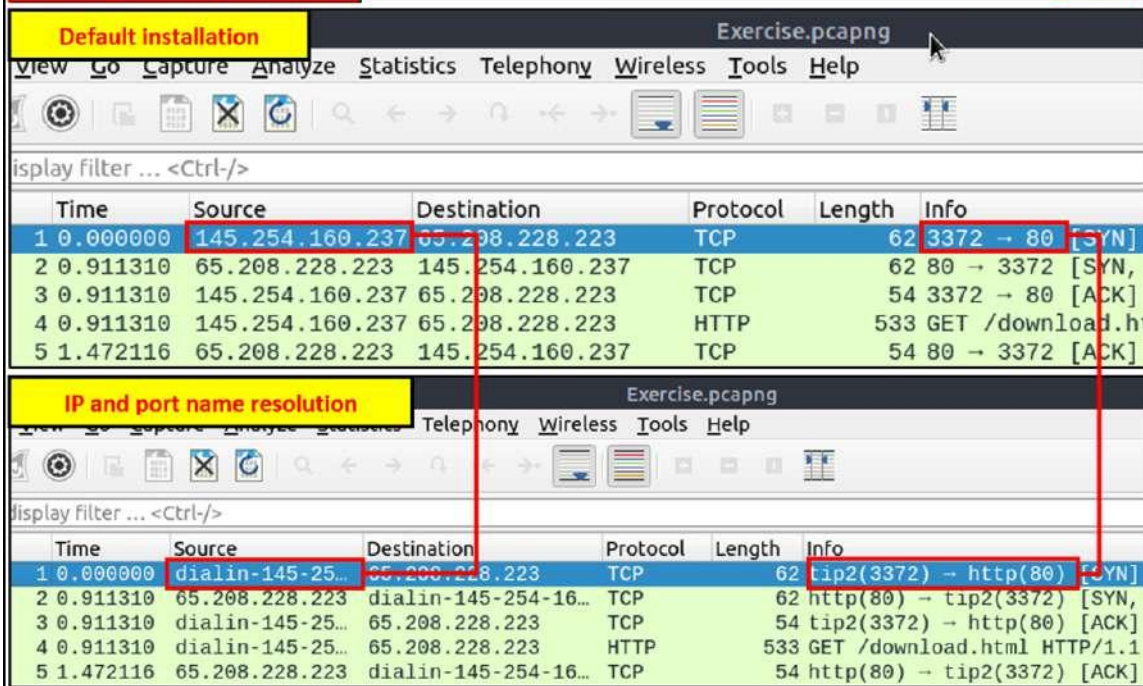
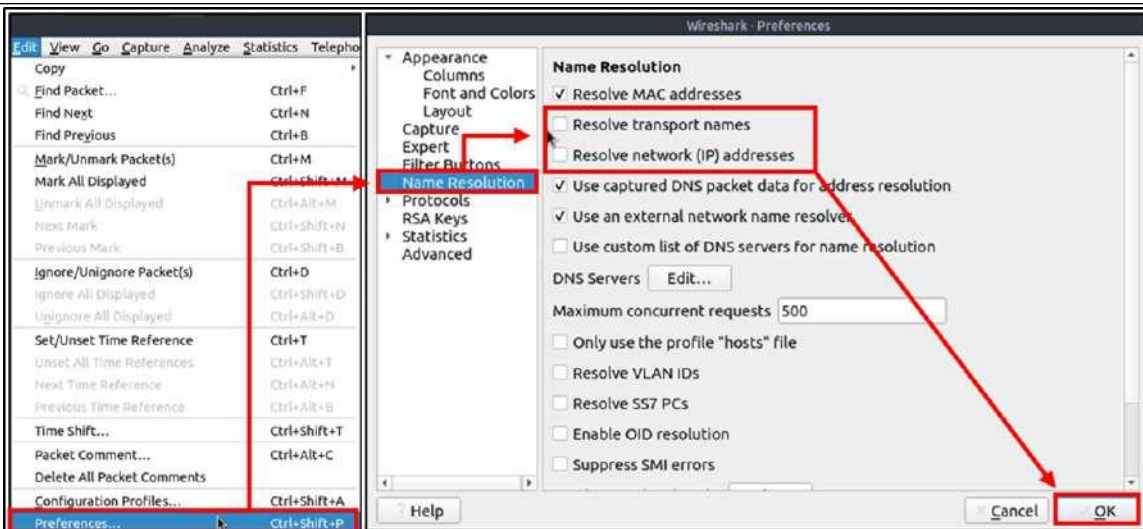
the "Statistics --> Endpoints" menu to view this info.

Wireshark also supports resolving MAC addresses to human-readable format using the manufacturer name assigned by IEEE. Note that this conversion is done through the first three bytes of the MAC address and only works for the known manufacturers. When you review the ethernet endpoints, you can activate this option with the **"Name resolution"** button in the lower-left corner of the endpoints window.

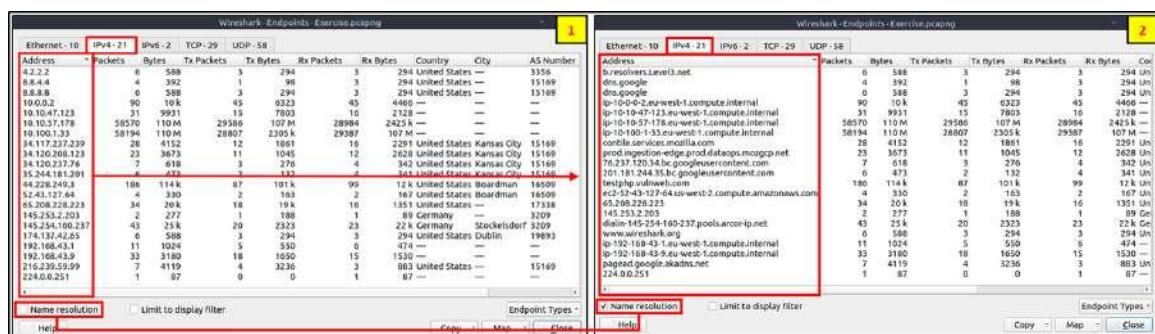


Name resolution is not limited only to MAC addresses. Wireshark provides IP and port name resolution options as well. **However, these options are not enabled by default.**

- If you want to use these functionalities, you need to activate them through the **"Edit --> Preferences --> Name Resolution"** menu. Once you enable IP and port name resolution, you will see the resolved IP address and port names in the packet list pane and also will be able to view resolved names in the "Conversations" and "Endpoints" menus as well.



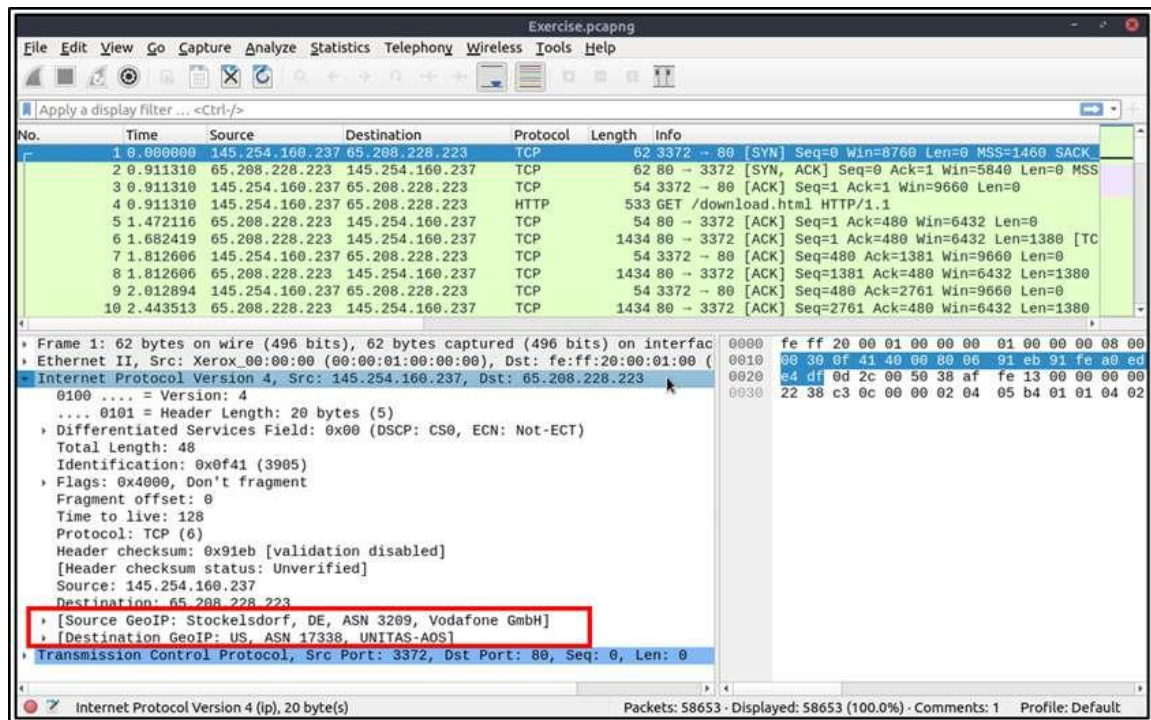
Endpoint menu view with name resolution:



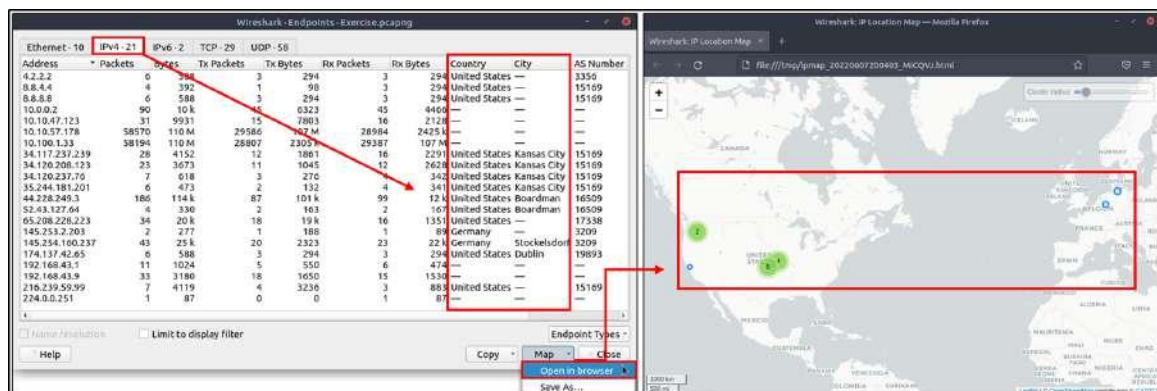
Besides name resolution, Wireshark also provides an **IP geolocation mapping** that helps analysts identify the map's source and destination

addresses.

But **this feature is not activated by default** and needs supplementary data like the **GeoIP database**. Currently, Wireshark supports MaxMind databases, and the latest versions of the Wireshark come configured MaxMind DB resolver. However, you still need MaxMind DB files and provide the database path to Wireshark by using the "Edit --> Preferences --> Name Resolution --> MaxMind database directories" menu. Once you download and indicate the path, Wireshark will automatically provide GeoIP information under the IP protocol details for the matched IP addresses.



Endpoints and GeoIP view.



IPv4 and IPv6

Up to here, almost all options provided information that contained both versions of the IP addresses. The statistics menu has two options for narrowing the statistics on packets containing a specific IP version. Thus, analysts can identify and list all events linked to specific IP versions in a single window and use it for the event of interest. You can use the **"Statistics --> IPvX Statistics"** menu to view this info.

The screenshot displays four Wireshark statistics windows for the file 'Exercise.pcapng'. Red arrows indicate the navigation path from the main interface to these statistics windows.

Wireshark - All Addresses - Exercise.pcapng

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
All Addresses	58646				0.0000	100%	2.2200	568415475.092
8.8.8.8	6				0.0000	0.01%	0.0100	285510489.972
8.8.4.4	4				0.0000	0.01%	0.0100	285510492.942
65.208.228.223	34				0.0000	0.06%	0.0300	0.911
52.43.127.64	4				0.0000	0.01%	0.0300	568415469.634
44.228.249.3	186				0.0000	0.32%	0.5100	568415507.869
4.2.2.2	6				0.0000	0.01%	0.0200	285510497.037
35.244.181.201	7				0.0000	0.01%	0.0600	568415488.790
34.120.237.76	7				0.0000	0.01%	0.0400	568415492.778
34.120.208.123	23				0.0000	0.04%	0.1000	568415440.158
34.117.237.239	28				0.0000	0.05%	0.1600	568415467.779
224.0.0.251	1				0.0000	0.00%	0.0100	568415487.070
216.239.59.99	7				0.0000	0.01%	0.0300	3.916
192.168.43.9	33				0.0000	0.06%	0.0300	285510489.960
192.168.43.1	11				0.0000	0.02%	0.0200	285510489.960
174.137.42.65	6				0.0000	0.01%	0.0100	285510500.825
145.254.160.237	43				0.0000	0.07%	0.0400	2.554
145.253.2.203	2				0.0000	0.00%	0.0100	2.554
10.100.1.33	58194				0.0000	99.23%	2.2200	568415475.092
10.10.57.178	58570				0.0000	99.87%	2.2200	568415475.092
10.10.47.123	31				0.0000	0.05%	0.1100	568415444.167
10.0.0.2	90				0.0000	0.15%	0.1600	568415439.987

Wireshark - IP Protocol Types - Exercise.pcapng

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
IP Protocol Types	58646				0.0000	100%	2.2200	568415475.092
UDP	104				0.0000	0.18%	0.1600	568415439.987
TCP	58520				0.0000	99.79%	2.2200	568415475.092
NONE	22				0.0000	0.04%	0.0200	285510497.037

Wireshark - Destinations and Ports - Exercise.pcapng

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
145.254.160.237	23				0.0000	0.04%	0.0200	2.554
UDP	1				0.0000	4.35%	0.0100	2.914
3009	1				0.0000	100.00%	0.0100	2.914
TCP	22				0.0000	95.65%	0.0200	2.554
3372	18				0.0000	81.82%	0.0200	2.554
3371	4				0.0000	18.18%	0.0200	3.916
145.253.2.203	1				0.0000	0.00%	0.0100	2.554
UDP	1				0.0000	100.00%	0.0100	2.554
53	1				0.0000	100.00%	0.0100	2.554
TCP	29387				0.0000	50.11%	1.1100	568415475.093
48924	13987				0.0000	100.00%	1.1100	568415475.093
43514	15400				0.0000	47.60%	0.8000	568415474.560
10.10.57.178	28984				0.0000	52.40%	0.8300	568415468.711
UDP	45				0.0000	0.16%	0.0800	568415439.988
59621	1				0.0000	2.22%	0.0100	568415439.988
58648	1				0.0000	2.22%	0.0100	568415439.997
58212	1				0.0000	2.22%	0.0100	568415508.025
57199	1				0.0000	2.22%	0.0100	568415439.992
56919	1				0.0000	2.22%	0.0100	568415438.276
56914	1				0.0000	2.22%	0.0100	568415496.706
56712	1				0.0000	2.22%	0.0100	568415520.218
56495	1				0.0000	2.22%	0.0100	568415507.387
54401	1				0.0000	2.22%	0.0100	568415444.331

Wireshark - Source and Destination Addresses - Exercise.pcapng

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst start
Source IPv4 Addresses	58646				0.0000	100%	2.2200	568415475.092
8.8.8.8	3				0.0000	0.01%	0.0100	285510490.464
8.8.4.4	1				0.0000	0.00%	0.0100	285510494.282
65.208.228.223	18				0.0000	0.03%	0.0200	2.554
52.43.127.64	2				0.0000	0.00%	0.0100	568415469.634
44.228.249.3	87				0.0000	0.15%	0.2800	568415507.869
4.2.2.2	3				0.0000	0.01%	0.0100	285510497.101
35.244.181.201	2				0.0000	0.00%	0.0200	568415488.792
34.120.237.76	3				0.0000	0.01%	0.0200	568415492.786
34.120.208.123	11				0.0000	0.02%	0.0600	568415440.158
34.117.237.239	12				0.0000	0.02%	0.0600	568415467.786
216.239.59.99	4				0.0000	0.01%	0.0200	3.916
192.168.43.9	18				0.0000	0.03%	0.0200	285510489.960
192.168.43.1	5				0.0000	0.01%	0.0100	285510489.960
174.137.42.65	3				0.0000	0.01%	0.0100	285510501.395
145.254.160.237	20				0.0000	0.03%	0.0200	0.911
145.253.2.203	1				0.0000	0.00%	0.0100	2.914
10.100.1.33	28807				0.0000	49.12%	1.2100	568415512.371
10.10.57.178	29586				0.0000	50.43%	1.1100	568415475.093
10.10.47.123	15				0.0000	0.03%	0.0500	568415439.936
10.0.0.2	45				0.0000	0.08%	0.0800	568415439.988
Destination IPv4 Addresses	58646				0.0000	100%	2.2200	568415475.092
8.8.8.8	3				0.0000	0.01%	0.0100	285510492.942
8.8.4.4	1				0.0000	0.01%	0.0100	285510492.942
65.208.228.223	16				0.0000	0.03%	0.0200	0.911
52.43.127.64	2				0.0000	0.00%	0.0200	568415469.634
44.228.249.3	90				0.0000	0.17%	0.2300	568415492.778
4.2.2.2	3				0.0000	0.01%	0.0100	285510497.037
35.244.181.201	4				0.0000	0.01%	0.0400	568415488.790
34.120.237.76	4				0.0000	0.01%	0.0200	568415492.776
34.120.208.123	12				0.0000	0.02%	0.0600	568415439.991
34.117.237.239	16				0.0000	0.03%	0.1000	568415467.779
224.0.0.251	1				0.0000	0.00%	0.0100	568415487.070
216.239.59.99	3				0.0000	0.01%	0.0100	2.584

DNS

This option breaks down all DNS packets from the capture file and helps analysts view the findings in a tree view based on packet counters and percentages of the DNS protocol. Thus analysts can view the DNS service's overall usage, including rcode, opcode, class, query type, service and query stats and use it for the event of interest. You can use the **"Statistics --> DNS"** menu to view this info.

The image shows the Wireshark Statistics window for the file 'Exercise.pcapng'. The left pane lists various statistics categories, with 'DNS' selected. The right pane displays the DNS statistics in a tree view. A red box highlights the 'Query Stats' section, which includes the following data:

Topic / Item	Count	Average	Min val	Max val	Rate (ms)	Percent	Burst rate	Burst
Query Stats	0				0.0000	100%	-	-
Qname Len	52	27.19	9	55	0.0000	-	0.0800	5684
Label Stats	0				0.0000	-	-	-
4th Level or more	31				0.0000	-	0.0800	5684
3rd Level	20				0.0000	-	0.0400	5684
2nd Level	1				0.0000	-	0.0100	5684
1st Level	0				0.0000	-	-	-
Payload size	103	75.38	35	282	0.0000	100%	0.1600	5684

HTTP

This option breaks down all HTTP packets from the capture file and helps analysts view the findings in a tree view based on packet counters and percentages of the HTTP protocol. Thus analysts can view the HTTP service's overall usage, including request and response codes and the original requests. You can use the **"Statistics --> HTTP"** menu to view this info.

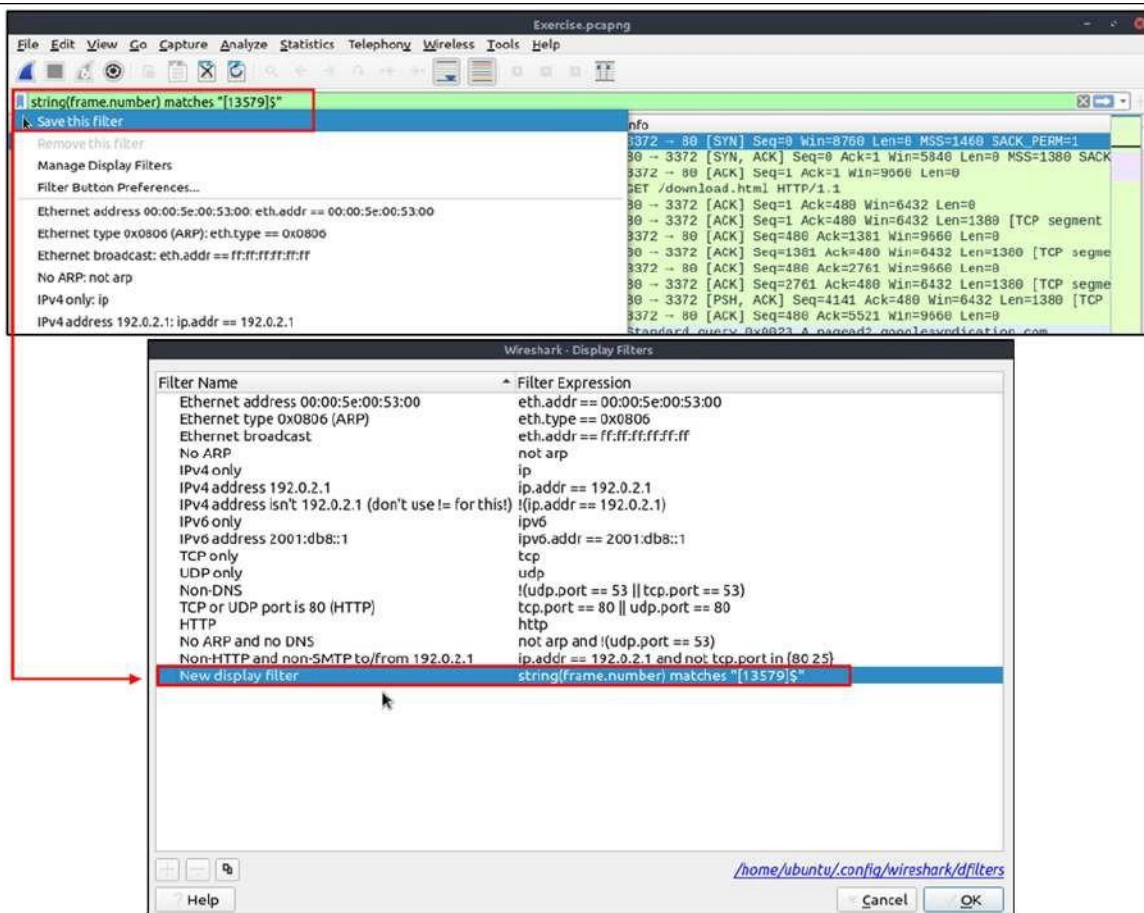
The screenshot displays the Wireshark interface with the following windows:

- Statistics:** Shows the protocol hierarchy with HTTP selected.
- Packet Counter:** Displays a table of statistics for various topics, including HTTP packets, response packets, and request packets.
- Load Distribution:** Shows the distribution of requests by host, including www.etherreal.com and testphp.vulnweb.com.
- Requests:** Lists individual HTTP requests by host, including various image and document requests.
- Request Sequences:** Lists the sequence of requests, including a sequence of image requests and a document request.

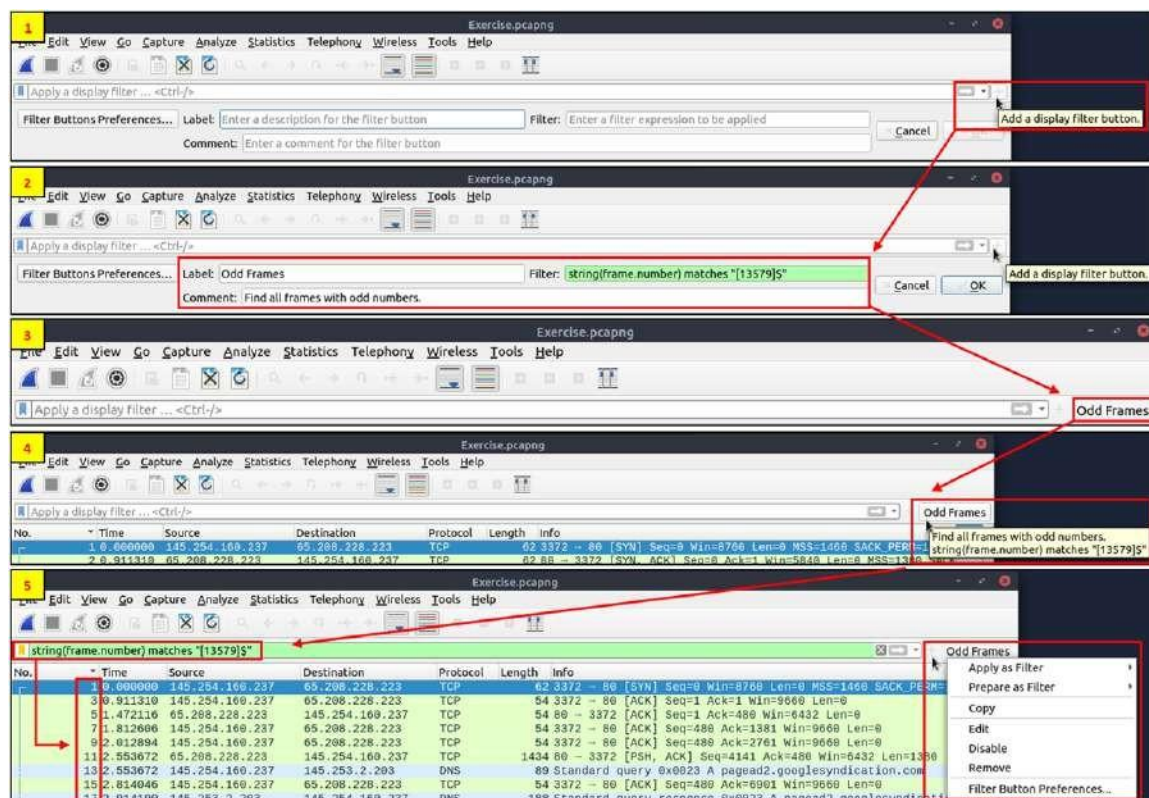
Bookmarks and Filtering Buttons

We've covered different types of filtering options, operators and functions. It is time to create filters and save them as bookmarks and buttons for later usage. As mentioned in the previous task, the filter toolbar has a filter bookmark section to save user-created filters, which helps analysts re-use favourite/complex filters with a couple of clicks. Similar to bookmarks, you can create filter buttons ready to apply with a single click.

Creating and using bookmarks.

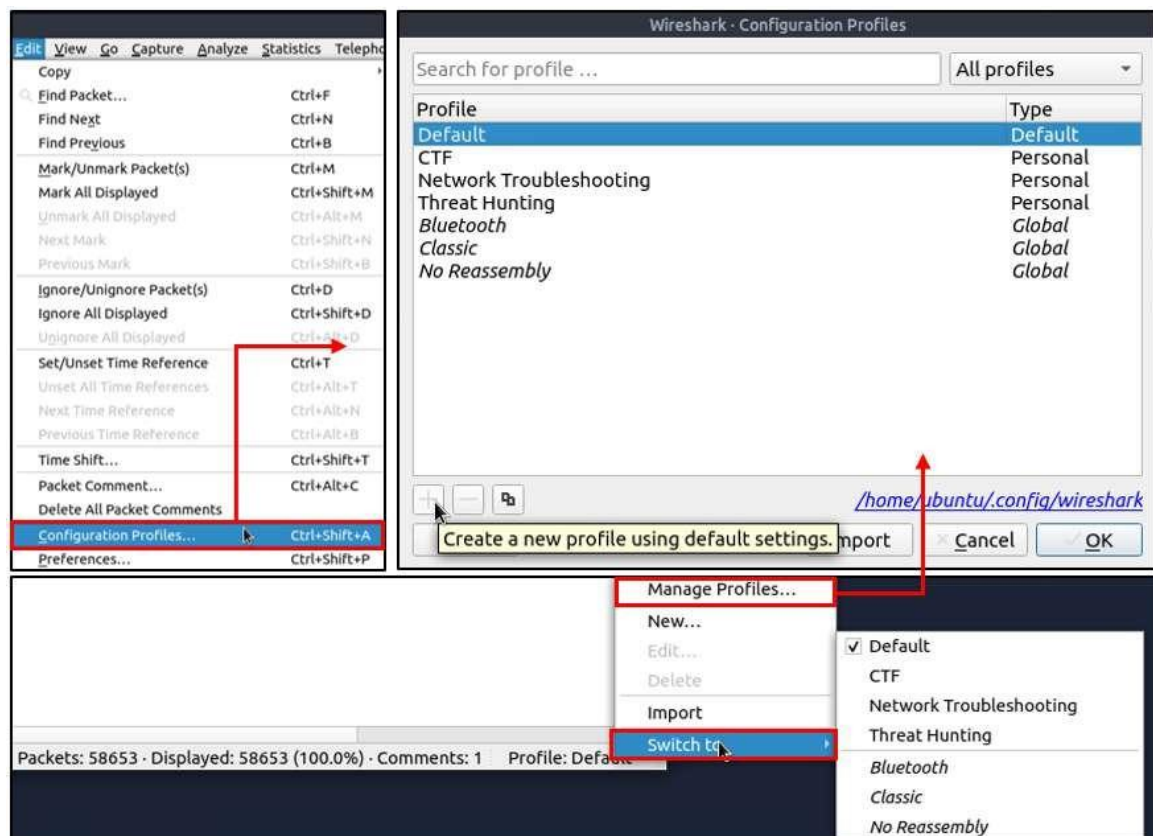


Creating and using display filter buttons.



Profiles

Wireshark is a multifunctional tool that helps analysts to accomplish in-depth packet analysis. As we covered during the room, multiple preferences need to be configured to analyse a specific event of interest. It is cumbersome to re-change the configuration for each investigation case, which requires a different set of colouring rules and filtering buttons. This is where Wireshark profiles come into play. You can create multiple profiles for different investigation cases and use them accordingly. You can use the **"Edit --> Configuration Profiles"** menu or the **"lower right bottom of the status bar --> Profile"** section to create, modify and change the profile configuration.



Packet Filtering

Capture Filters	This type of filter is used to save only a specific part of the traffic. It is set before capturing traffic and not changeable during the capture.
Display Filters	This type of filter is used to investigate packets by reducing the number of visible packets, and it is changeable during the capture.

Note: You cannot use the display filter expressions for capturing traffic and vice versa.

The typical use case is capturing everything and filtering the packets according to the event of interest. Only experienced professionals use capture filters and sniff traffic. This is why Wireshark supports more protocol types in display filters.

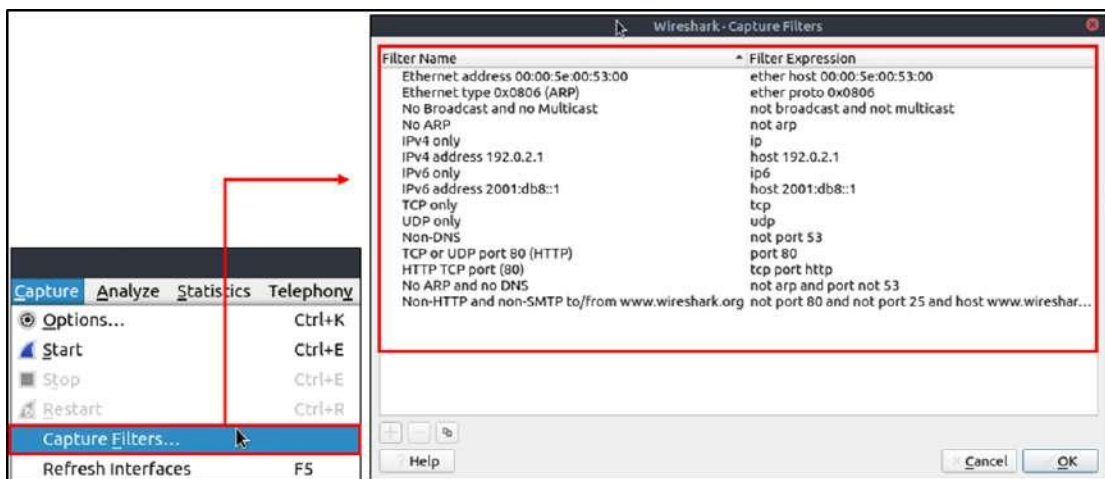
Capture Filter Syntax

These filters use byte offsets hex values and masks with boolean operators, and it is not easy to understand/predict the filter's purpose at first glance. The base syntax is explained below:

- **Scope:** host, net, port and portrange.
- **Direction:** src, dst, src or dst, src and dst,
- **Protocol:** ether, wlan, ip, ip6, arp, rarp, tcp and udp.
- **Sample filter to capture port 80 traffic:**

`tcp port 80`

You can read more on capture filter syntax from [here](#) and [here](#). A quick reference is available under the "Capture --> Capture Filters" menu.



Display Filter Syntax

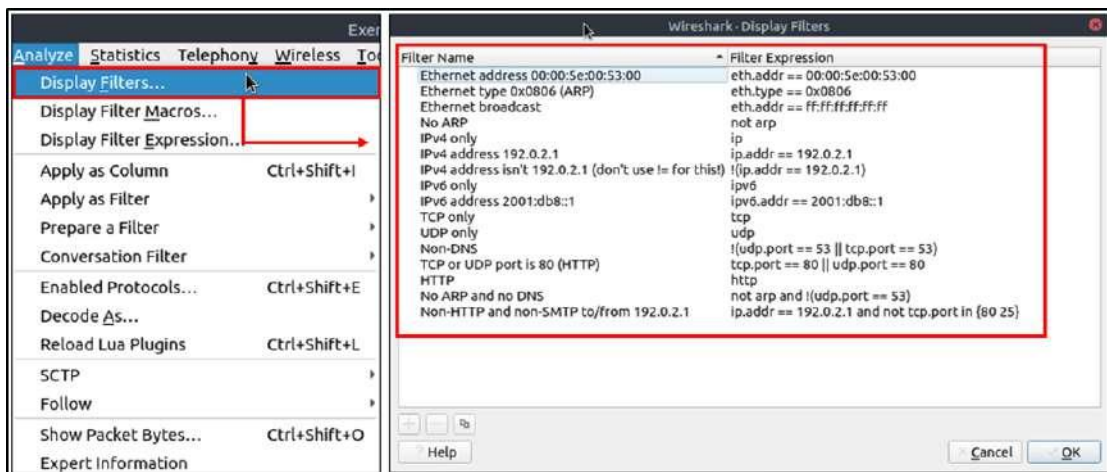
This is Wireshark's most powerful feature. It supports 3000 protocols and allows conducting packet-level searches under the protocol breakdown. The official "[Display Filter Reference](#)" provides all supported protocols breakdown for filtering.

- **Sample filter to capture port 80 traffic:**

`tcp.port == 80`

Wireshark has a built-in option (Display Filter Expression) that stores all supported protocol structures to help analysts create display filters. We will cover the "Display Filter Expression" menu later. Now let's understand the

fundamentals of the display filter operations. A quick reference is available under the **"Analyse --> Display Filters"** menu.



Comparison Operators

You can create display filters by using different comparison operators to find the event of interest. The primary operators are shown in the table below.

English	C-Like	Description	Example
eq	<code>==</code>	Equal	<code>ip.src == 10.10.10.100</code>
ne	<code>!=</code>	Not equal	<code>ip.src != 10.10.10.100</code>
gt	<code>></code>	Greater than	<code>ip.ttl > 250</code>
lt	<code><</code>	Less Than	<code>ip.ttl < 10</code>
ge	<code>>=</code>	Greater than or equal to	<code>ip.ttl >= 0xFA</code>
le	<code><=</code>	Less than or equal to	<code>ip.ttl <= 0xA</code>

Note: Wireshark supports decimal and hexadecimal values in filtering. You can use any format you want according to the search you will conduct.

Logical Expressions

Wireshark supports boolean syntax. You can create display filters by using logical operators as well.

English	C-Like	Description	Example
and	<code>&&</code>	Logical AND	<code>(ip.src == 10.10.10.100) AND (ip.src == 10.10.10.111)</code>
or	<code> </code>	Logical OR	<code>(ip.src == 10.10.10.100) OR (ip.src == 10.10.10.111)</code>

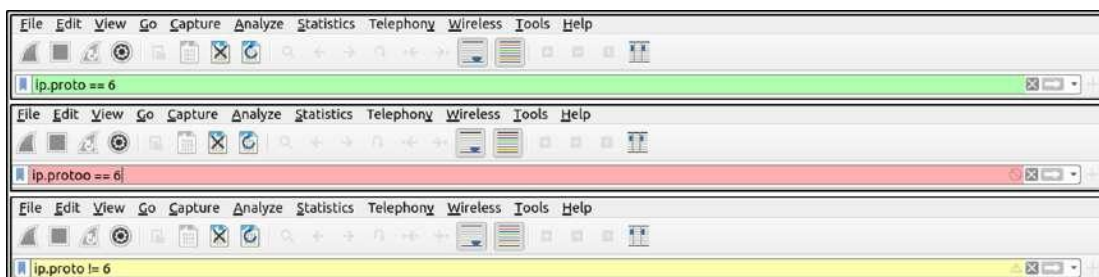
not	!	Logical NOT	<pre>!(ip.src == 10.10.10.222)</pre> Note: Usage of <code>!=value</code> is deprecated; using it could provide inconsistent results. Using the <code>!(value)</code> style is suggested for more consistent results.
------------	----------	-------------	--

Packet Filter Toolbar

The filter toolbar is where you create and apply your display filters. It is a smart toolbar that helps you create valid display filters with ease. Before starting to filter packets, here are a few tips:

- Packet filters are defined in lowercase.
- Packet filters have an autocomplete feature to break down protocol details, and each detail is represented by a "dot".
- Packet filters have a three-colour representation explained below.

Green	Valid filter
Red	Invalid filter
Yellow	Warning filter. This filter works, but it is unreliable, and it is suggested to change it with a valid filter.



Protocol Filters

As mentioned in the previous task, Wireshark supports 3000 protocols and allows packet-level investigation by filtering the protocol fields. This task shows the creation and usage of filters against different protocol fields.

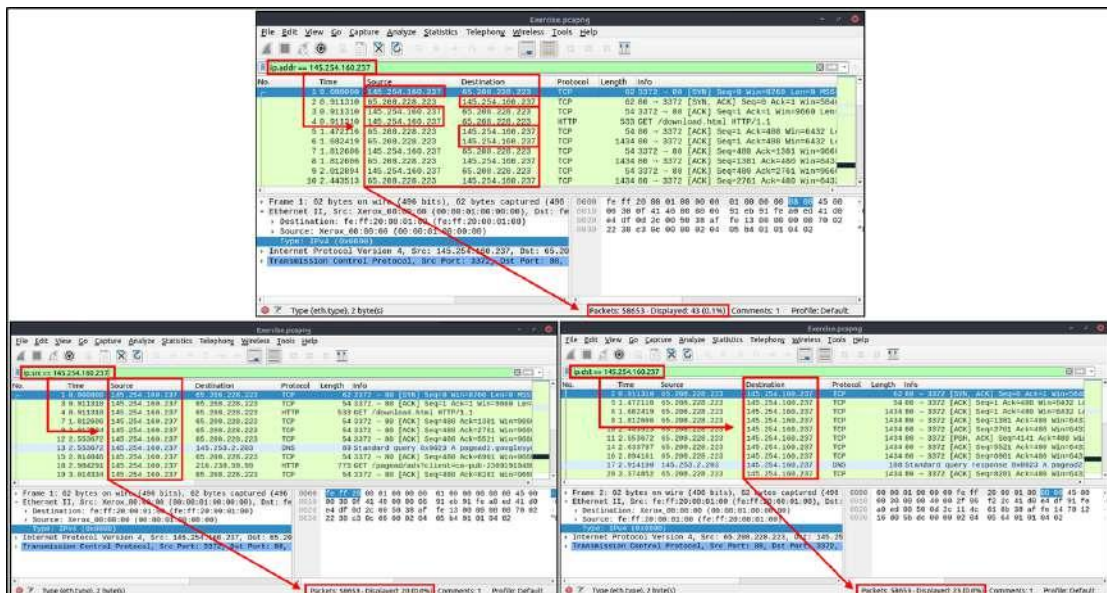
IP Filters

IP filters help analysts filter the traffic according to the IP level information from the packets (Network layer of the OSI model). This is one of the most commonly used filters in Wireshark. These filters filter network-level

information like IP addresses, version, time to live, type of service, flags, and checksum values.

The common filters are shown in the given table.

Filter	Description
ip	Show all IP packets.
ip.addr == 10.10.10.111	Show all packets containing IP address 10.10.10.111.
ip.addr == 10.10.10.0/24	Show all packets containing IP addresses from 10.10.10.0/24 subnet.
ip.src == 10.10.10.111	Show all packets originated from 10.10.10.111
ip.dst == 10.10.10.111	Show all packets sent to 10.10.10.111
ip.addr vs ip.src/ip.dst	Note: The ip.addr filters the traffic without considering the packet direction. The ip.src/ip.dst filters the packet depending on the packet direction.



TCP

TCP filters help analysts filter the traffic according to protocol-level information from the packets (Transport layer of the OSI model). These filters filter transport protocol level information like source and destination ports, sequence number, acknowledgement number, windows size, timestamps, flags, length and protocol errors.

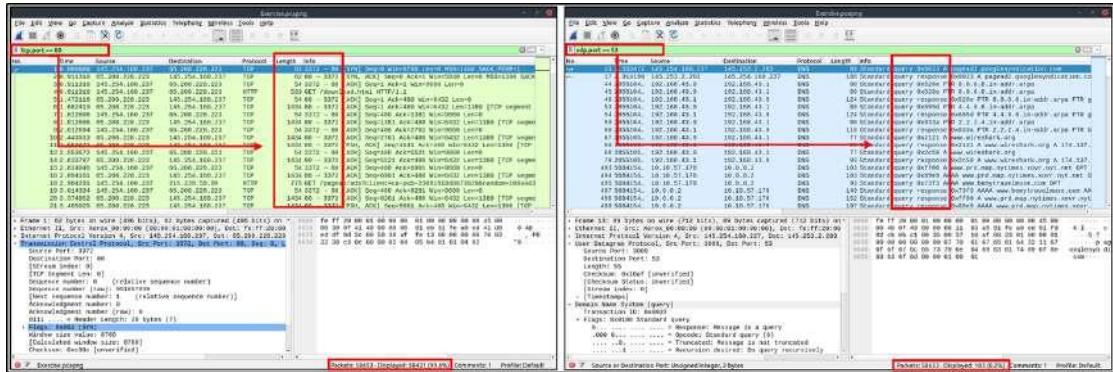
Filter	Description	Filter	Expression
tcp.port == 80	Show all TCP packets with port 80	udp.port == 53	Show all UDP packets with port 53
tcp.srcport == 1234	Show all TCP packets originating from port 1234	udp.srcport == 1234	Show all UDP packets originating from port 1234

tcp.dstport == 80

Show all TCP packets sent to port 80

udp.dstport == 5353

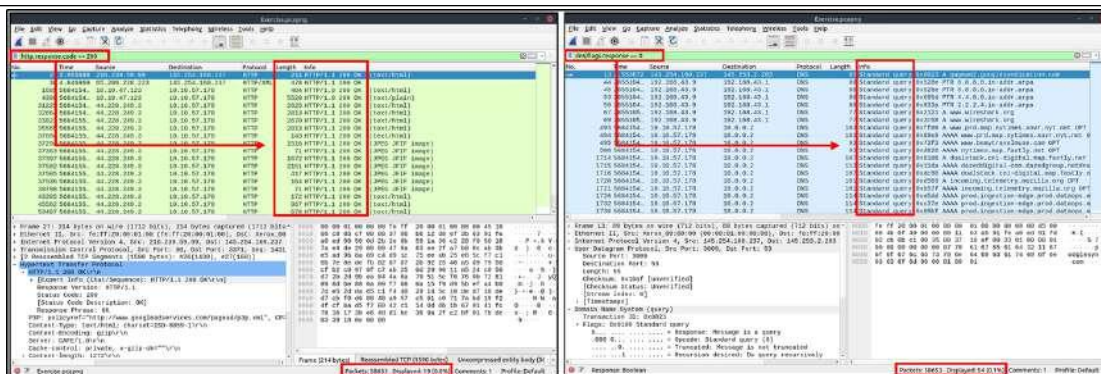
Show all UDP packets sent to port 5353



Application Level Protocol Filters | and DNS

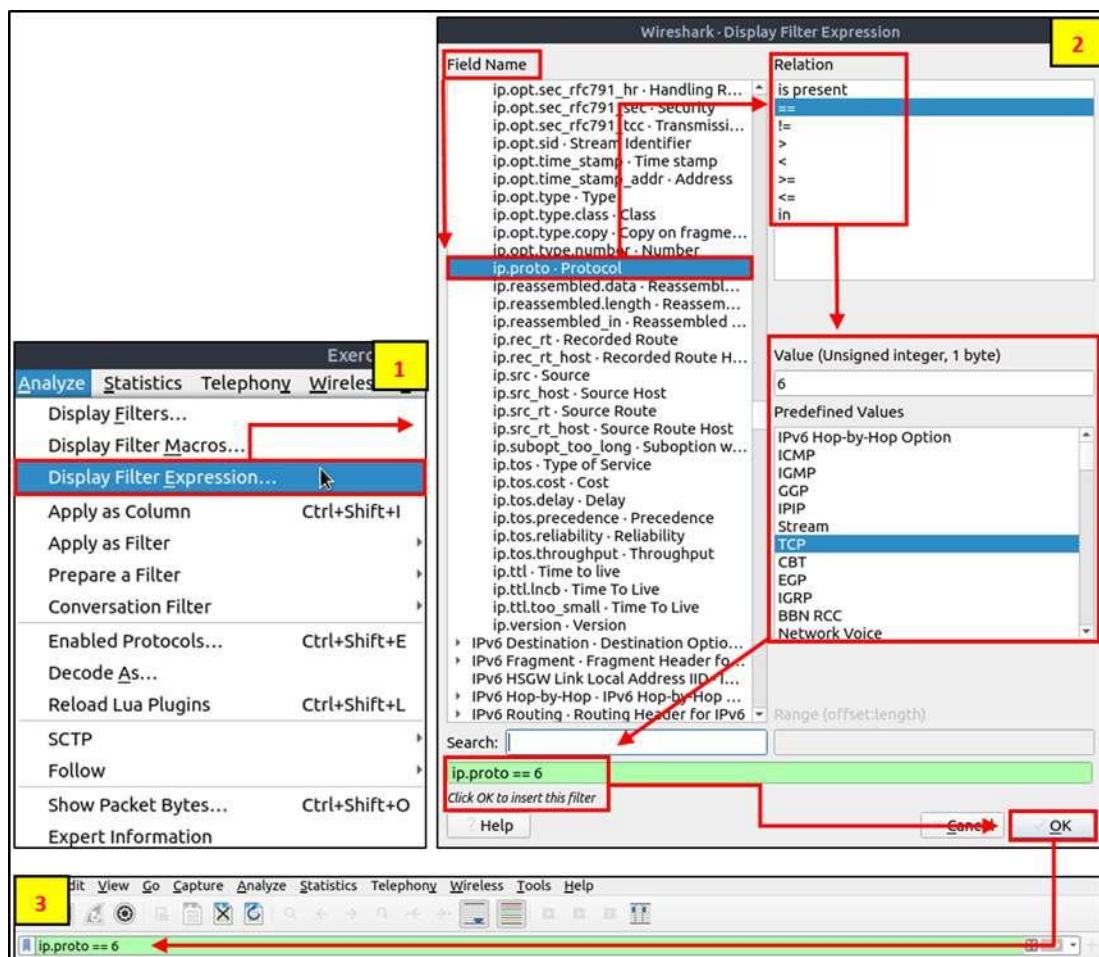
Application-level protocol filters help analysts filter the traffic according to application protocol level information from the packets (Application layer of the OSI model). These filters filter application-specific information, like payload and linked data, depending on the protocol type.

Filter	Description	Filter	Description
http	Show all HTTP packets	dns	Show all DNS packets
http.response.code == 200	Show all packets with HTTP response code "200"	dns.flags.response == 0	Show all DNS requests
http.request.method == "GET"	Show all HTTP GET requests	dns.flags.response == 1	Show all DNS responses
http.request.method == "POST"	Show all HTTP POST requests	dns.qry.type == 1	Show all DNS "A" records



Display Filter Expressions

- Wireshark has a built-in option (Display Filter Expression) that stores all supported protocol structures to help analysts create display filters. When an analyst can't recall the required filter for a specific protocol or is unsure about the assignable values for a filter, the Display Filter Expressions menu provides an easy-to-use display filter builder guide. It is available under the "Analyze --> Display Filter Expression" menu.

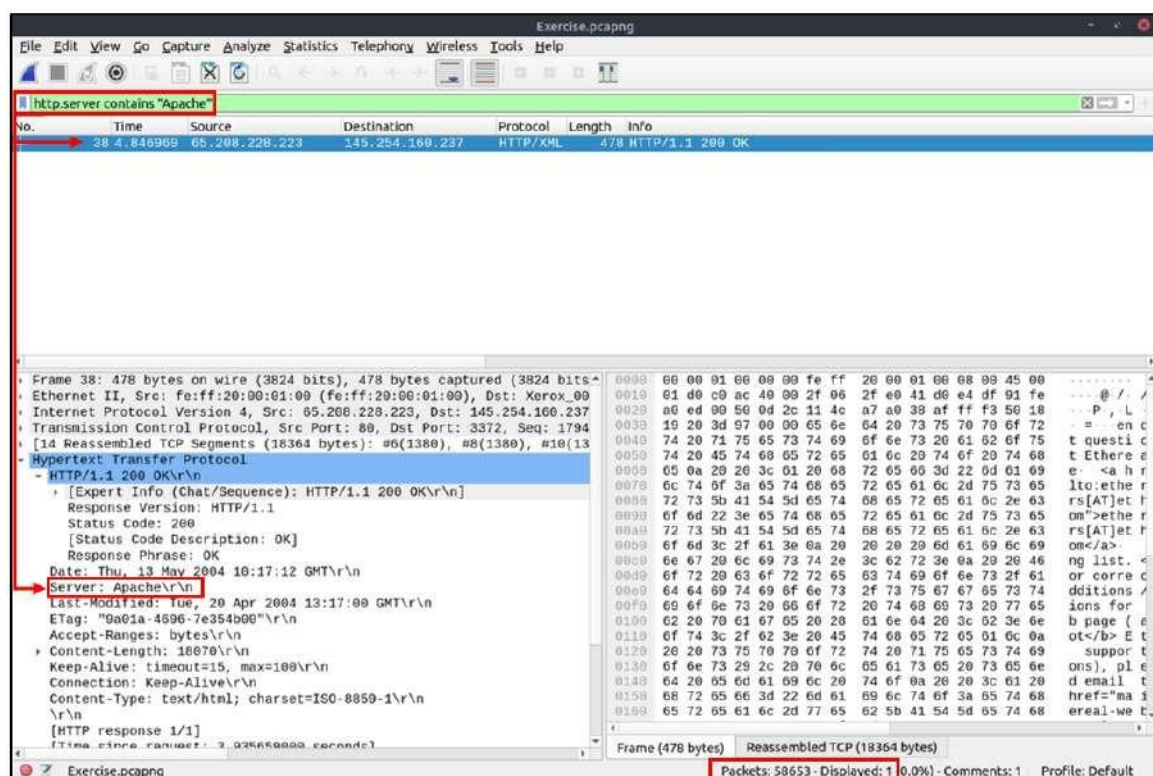


Advanced Filtering

So far, you have learned the basics of packet filtering operations. Now it is time to focus on specific packet details for the event of interest. Besides the operators and expressions covered in the previous room, Wireshark has advanced operators and functions. These advanced filtering options help the analyst conduct an in-depth analysis of an event of interest.

Filter: "contains"

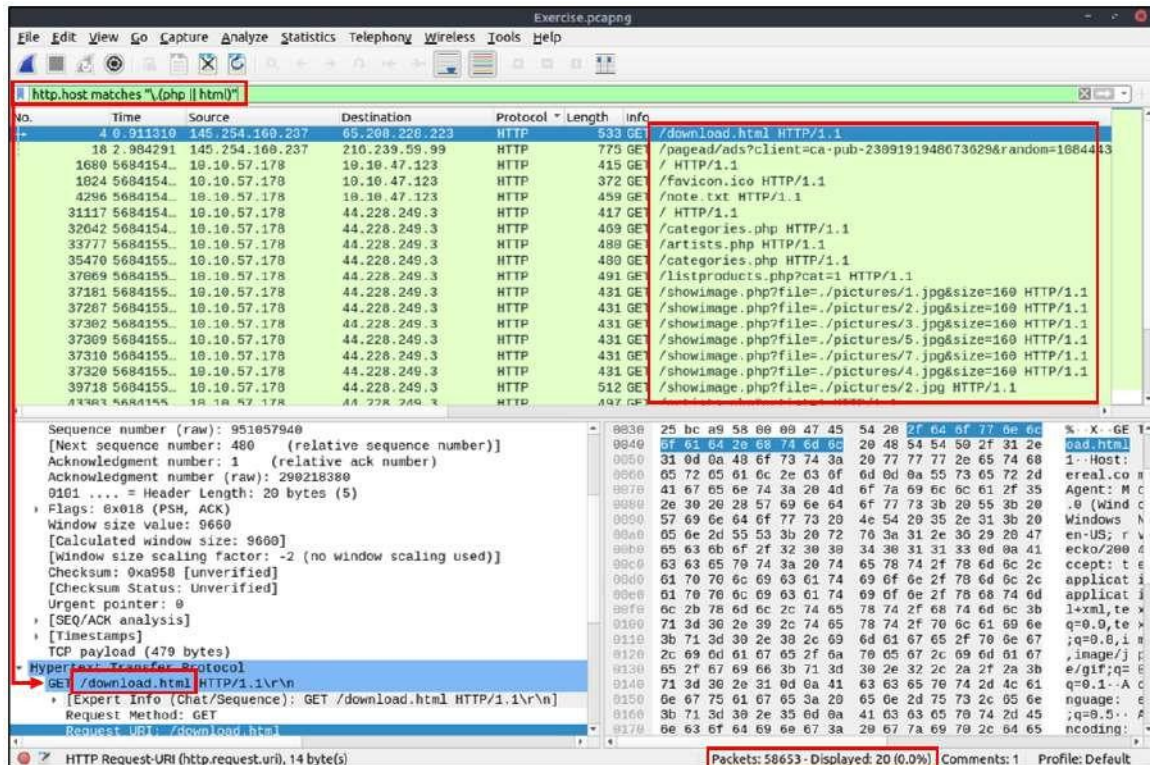
Filter	contains
Type	Comparison Operator
Description	Search a value inside packets. It is case-sensitive and provides similar functionality to the "Find" option by focusing on a specific field.
Example	Find all "Apache" servers.
Workflow	List all HTTP packets where packets' "server" field contains the "Apache" keyword.
Usage	<code>http.server contains "Apache"</code>



Filter: "matches"

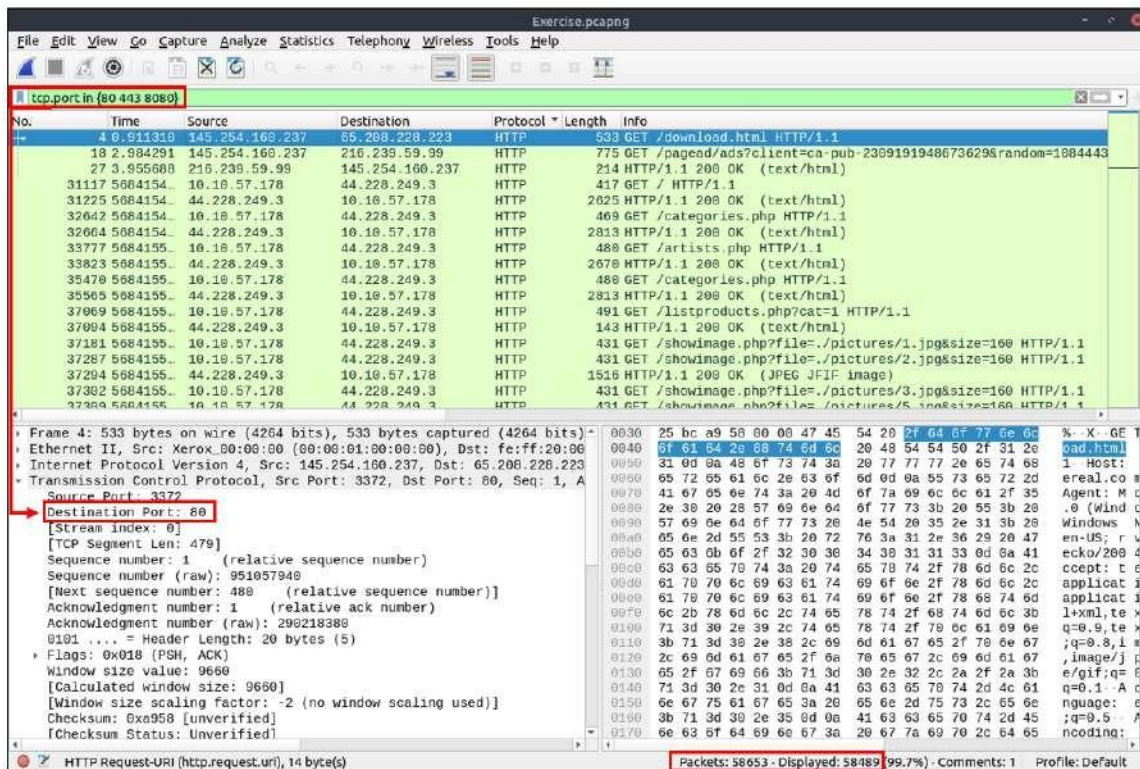
Filter	matches
Type	Comparison Operator
Description	Search a pattern of a regular expression. It is case insensitive, and complex queries have a margin of error.
Example	Find all .php and .html pages.

Workflow	List all HTTP packets where packets' "host" fields match keywords ".php" or ".html".
Usage	<code>http.host matches "\.(php html)"</code>



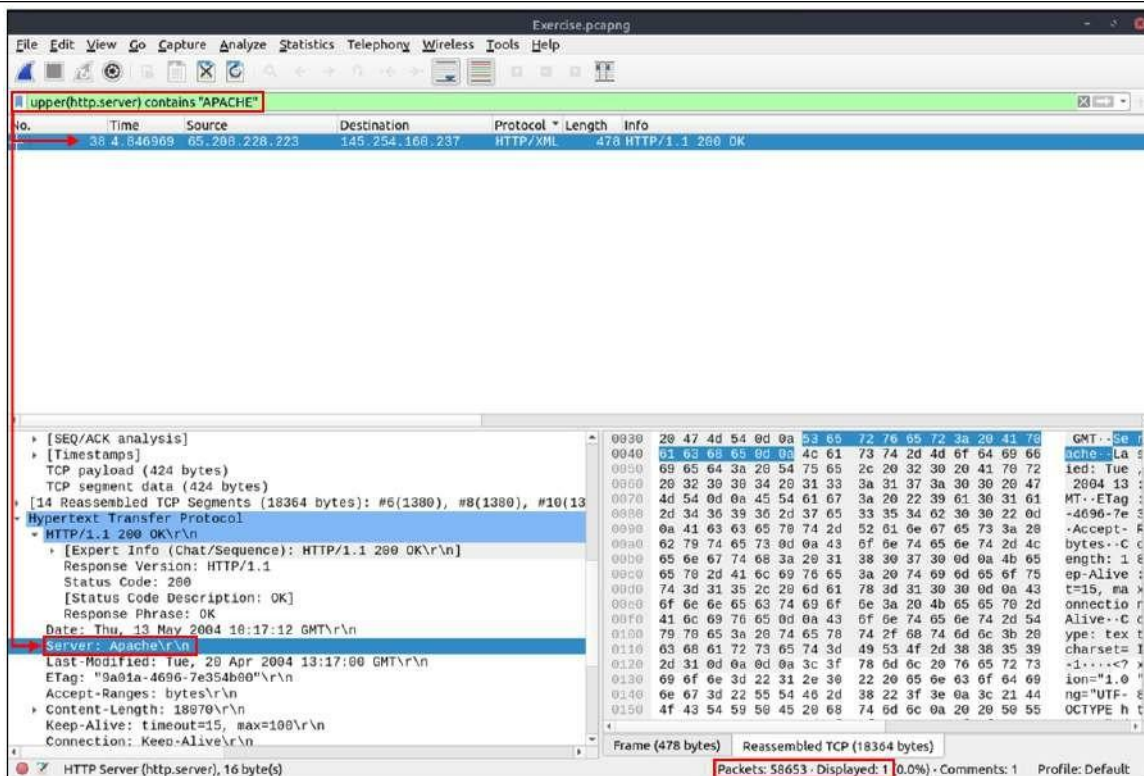
Filter: "in"

Filter	in
Type	Set Membership
Description	Search a value or field inside of a specific scope/range.
Example	Find all packets that use ports 80, 443 or 8080.
Workflow	List all TCP packets where packets' "port" fields have values 80, 443 or 8080.
Usage	<code>tcp.port in {80 443 8080}</code>



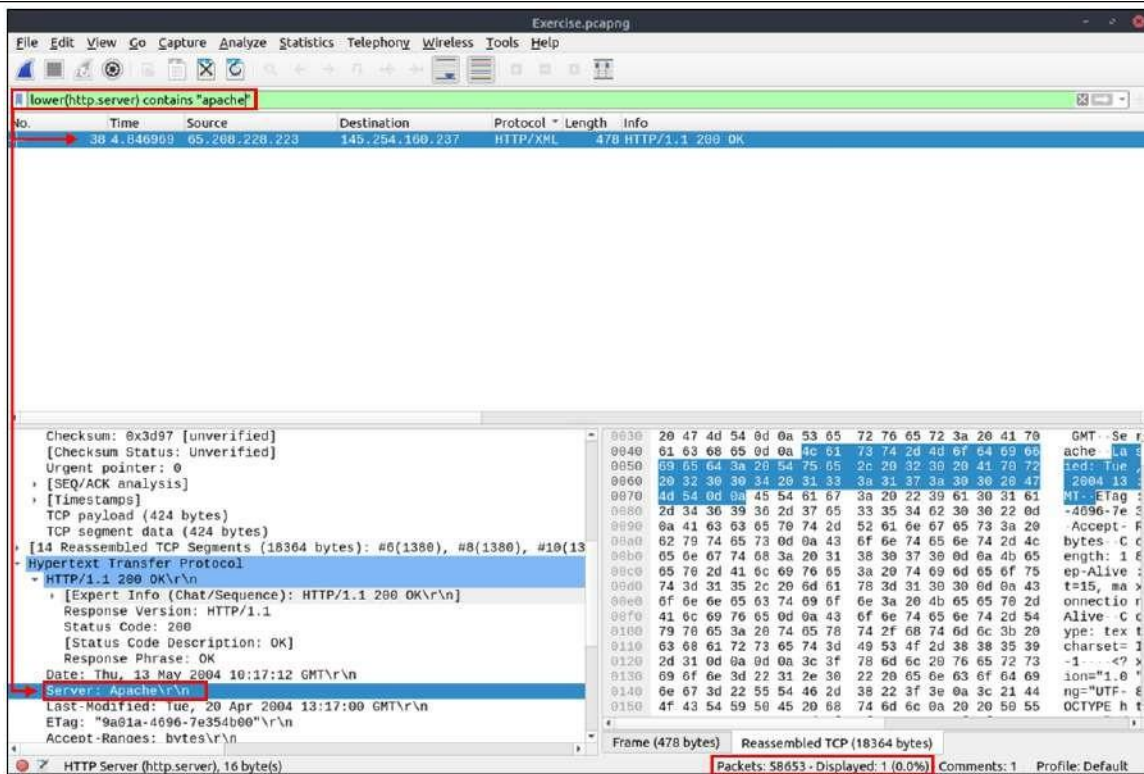
Filter: "upper"

Filter	upper
Type	Function
Description	Convert a string value to uppercase.
Example	Find all "APACHE" servers.
Workflow	Convert all HTTP packets' "server" fields to uppercase and list packets that contain the "APACHE" keyword.
Usage	<code>upper(http.server) contains "APACHE"</code>



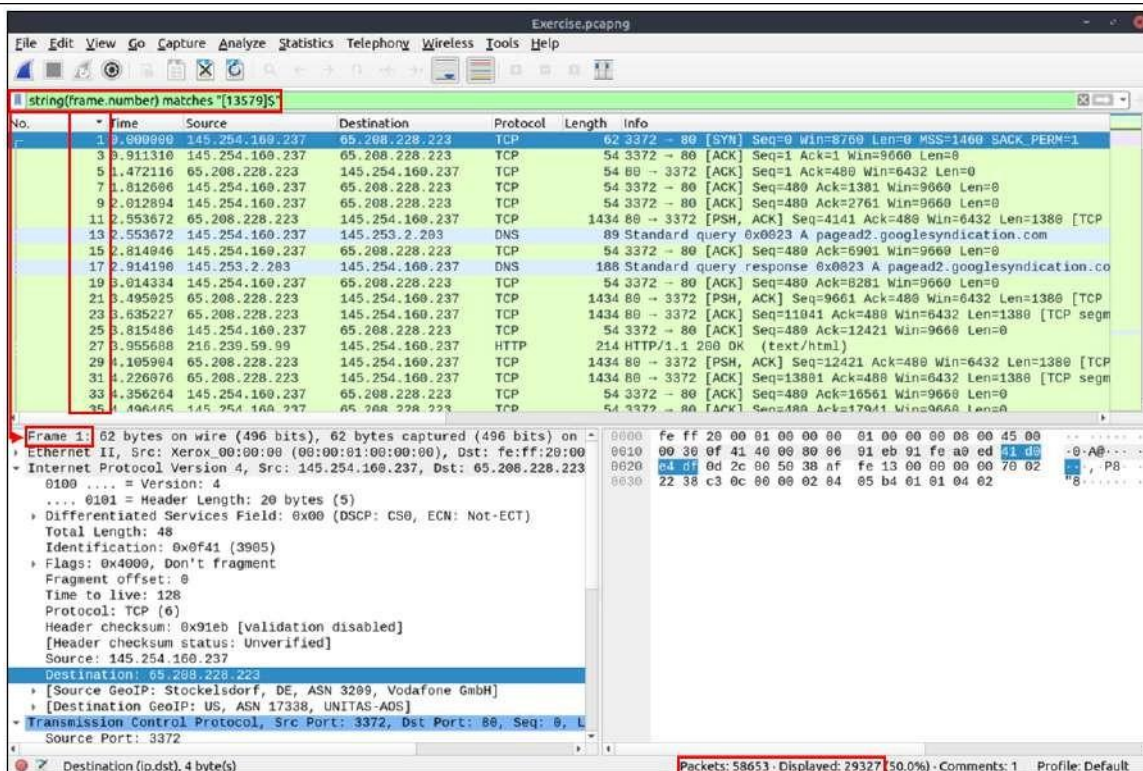
Filter: "lower"

Filter	lower
Type	Function
Description	Convert a string value to lowercase.
Example	Find all "apache" servers.
Workflow	Convert all HTTP packets' "server" fields info to lowercase and list packets that contain the "apache" keyword.
Usage	<code>lower(http.server) contains "apache"</code>



Filter: "string"

Filter	string
Type	Function
Description	Convert a non-string value to a string.
Example	Find all frames with odd numbers.
Workflow	Convert all "frame number" fields to string values, and list frames end with odd values.
Usage	<code>string(frame.number) matches "[13579]\$"</code>



Wireshark: Traffic Analysis

Nmap Scans

Nmap is an industry-standard tool for mapping networks, identifying live hosts and discovering the services. As it is one of the most used network scanner tools, a security analyst should identify the network patterns created with it. This section will cover identifying the most common Nmap scan types.

- TCP connect scans • SYN scans
- UDP scans

It is essential to know how Nmap scans work to spot scan activity on the network. However, it is impossible to understand the scan details without using the correct filters. Below are the base filters to probe Nmap scan behaviour on the network.

TCP flags in a nutshell.

Notes	Wireshark Filters
Global search.	• tcp • udp
• Only SYN flag. • SYN flag is set. The rest of the bits are not important.	• tcp.flags == 2 • tcp.flags.syn == 1
• Only ACK flag. • ACK flag is set. The rest of the bits are not important.	• tcp.flags == 16 • tcp.flags.ack == 1

• Only SYN, ACK flags. • SYN and ACK are set. The rest of the bits are not important.	• <code>tcp.flags == 18</code> • <code>(tcp.flags.syn == 1) and (tcp.flags.ack == 1)</code>
• Only RST flag. • RST flag is set. The rest of the bits are not important.	• <code>tcp.flags == 4</code> • <code>tcp.flags.reset == 1</code>
• Only RST, ACK flags. • RST and ACK are set. The rest of the bits are not important.	• <code>tcp.flags == 20</code> • <code>(tcp.flags.reset == 1) and (tcp.flags.ack == 1)</code>
• Only FIN flag • FIN flag is set. The rest of the bits are not important.	• <code>tcp.flags == 1</code> • <code>tcp.flags.fin == 1</code>

TCP

TCP Connect Scan in a nutshell:

- Relies on the three-way handshake (needs to finish the handshake process).

Usually conducted with `nmap` command.

- Used by non-privileged users (only option for a non-root user).
- Usually has a windows size **larger than 1024 bytes** as the request expects some data due to the nature of the protocol.

Open TCP Port	Open TCP Port	Closed TCP Port
• SYN --> • <-- SYN, ACK • ACK -->	• SYN --> • <-- SYN, ACK • ACK --> • RST, ACK -->	• SYN --> • <-- RST, ACK

The images below show the three-way handshake process of the open and close TCP ports. Images and pcap samples are split to make the investigation easier and understand each case's details.

Open TCP port (Connect):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	36958 → 22 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 SACK_PERM=1 TSval=
2	0.000012250	10.10.47.123	10.10.60.7	TCP	22 → 36958 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=8961 SACK_P...
3	0.000209974	10.10.60.7	10.10.47.123	TCP	36958 → 22 [ACK] Seq=1 Ack=1 Win=62848 Len=0 TSval=1438758498 TSe...
4	0.000244154	10.10.60.7	10.10.47.123	TCP	36958 → 22 [RST, ACK] Seq=1 Ack=1 Win=62848 Len=0 TSval=143875849...

Closed TCP port (Connect):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	59934 → 21 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 SACK_PERM=1 TSval=
2	0.000005840	10.10.47.123	10.10.60.7	TCP	21 → 59934 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

The above images provide the patterns in isolated traffic. However, it is not always easy to spot the given patterns in big capture files. Therefore analysts need to use a generic filter to view the initial anomaly patterns, and then it will be easier to focus on a specific traffic point.

- The given filter shows the TCP Connect scan patterns in a capture file.

```
tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size > 1024
```

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	45830 → 135 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
2	0.000000130	10.10.60.7	10.10.47.123	TCP	33436 → 23 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
3	0.000012991	10.10.60.7	10.10.47.123	TCP	34242 → 1025 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
4	0.000013031	10.10.60.7	10.10.47.123	TCP	49110 → 8888 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
5	0.000013071	10.10.60.7	10.10.47.123	TCP	51038 → 443 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
11	0.000059761	10.10.60.7	10.10.47.123	TCP	36958 → 22 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
13	0.000110152	10.10.60.7	10.10.47.123	TCP	59934 → 21 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
14	0.000110252	10.10.60.7	10.10.47.123	TCP	50882 → 554 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...
17	0.000131072	10.10.60.7	10.10.47.123	TCP	59022 → 111 [SYN] Seq=0 Win=62727 Len=0 MSS=8961 S...

SYN Scans

TCP SYN Scan in a nutshell:

- Doesn't rely on the three-way handshake (no need to finish the handshake process).

Usually conducted with Used by `map -ss` command.

privileged users.

Usually have a size **less than or equal** to **1024** bytes as the request is not finished and it doesn't expect to receive data.

Open TCP Port

Close TCP Port

• SYN --> • <-- SYN,ACK • **RST-->** • SYN --> • <-- RST,ACK

Open TCP port (SYN):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	36044 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.000047361	10.10.47.123	10.10.60.7	TCP	22 → 36044 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=8961
3	0.000269174	10.10.60.7	10.10.47.123	TCP	36044 → 22 [RST] Seq=1 Win=0 Len=0

Closed TCP port (SYN):

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	36044 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.000070660	10.10.47.123	10.10.60.7	TCP	21 → 36044 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

The given filter shows the TCP SYN scan patterns in a capture file.

```
tcp.flags.syn==1 and tcp.flags.ack==0 and tcp.window_size <= 1024
```

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	TCP	36044 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.000000000	10.10.60.7	10.10.47.123	TCP	36044 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
3	0.000000240	10.10.60.7	10.10.47.123	TCP	36044 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
4	0.000000250	10.10.60.7	10.10.47.123	TCP	36044 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
5	0.000000290	10.10.60.7	10.10.47.123	TCP	36044 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
11	0.000079761	10.10.60.7	10.10.47.123	TCP	36044 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
13	0.000091302	10.10.60.7	10.10.47.123	TCP	36044 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
14	0.000091372	10.10.60.7	10.10.47.123	TCP	36044 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
15	0.000091412	10.10.60.7	10.10.47.123	TCP	36044 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

UDP

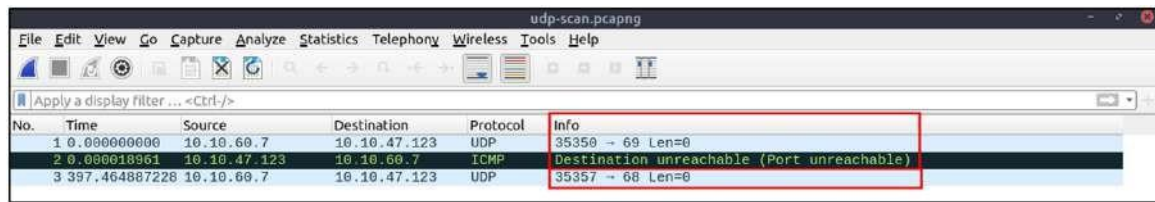
UDP Scan in a nutshell:

- Doesn't require a handshake process • No prompt for open ports
- ICMP error message for close ports

Usually conducted with command.

Open UDP Port	Closed UDP Port
• UDP packet -->	• UDP packet --> • ICMP Type 3, Code 3 message. (Destination unreachable, port unreachable)

Closed (port no 69) and open (port no 68) UDP ports:

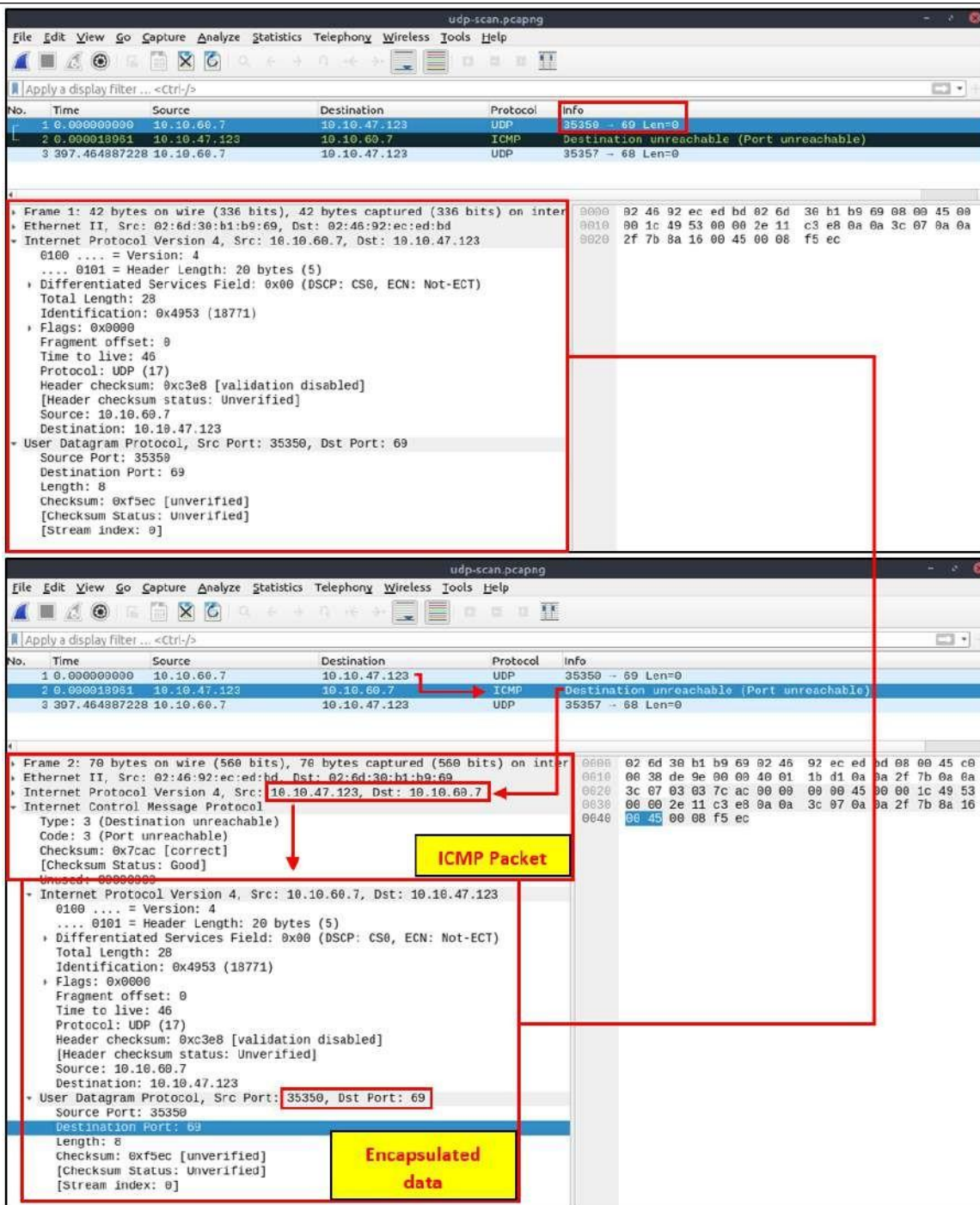


The image shows a Wireshark packet capture window titled 'udp-scan.pcapng'. The packet list pane shows three packets. The second packet, an ICMP error message, is highlighted with a red box. The packet details pane shows the ICMP section with the message 'Destination unreachable (Port unreachable)'.

No.	Time	Source	Destination	Protocol	Info
1	0.000000000	10.10.60.7	10.10.47.123	UDP	35350 → 69 Len=0
2	0.000018961	10.10.47.123	10.10.60.7	ICMP	Destination unreachable (Port unreachable)
3	337.464887228	10.10.60.7	10.10.47.123	UDP	35357 → 68 Len=0

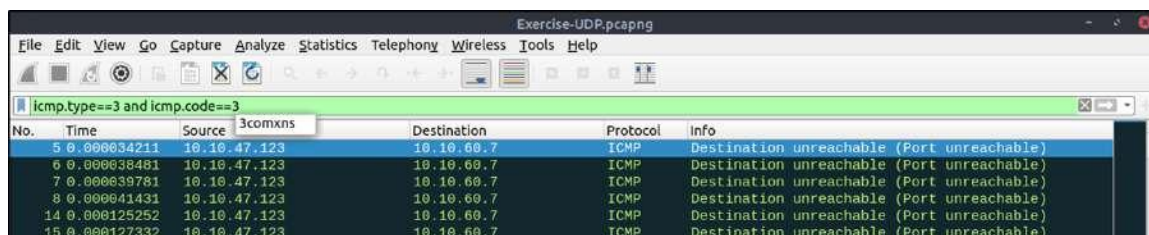
The above image shows that the closed port returns an ICMP error packet. No further information is provided about the error at first glance, so how can an analyst decide where this error message belongs?

- The ICMP error message uses the original request as encapsulated data to show the source/reason of the packet. Once you expand the ICMP section in the packet details pane, you will see the encapsulated data and the original request, as shown in the below image.



The given filter shows the UDP scan patterns in a capture file.

`icmp.type==3 and icmp.code==3`



ARP Poisoning & Man In The Middle!

- **ARP** protocol, or Address **R**esolution **P**rotocol (**ARP**), is the technology responsible for allowing devices to identify themselves on a network.
- Address Resolution Protocol Poisoning (also known as **ARP Spoofing** or Man In The Middle (MITM) attack) is a type of attack that involves network jamming/manipulating by sending malicious ARP packets to the default gateway. The ultimate aim is to manipulate the "**IP to MAC address table**" and sniff the traffic of the target host.

There are a variety of tools available to conduct ARP attacks. However, the mindset of the attack is static, so it is easy to detect such an attack by knowing the ARP protocol workflow and Wireshark skills.

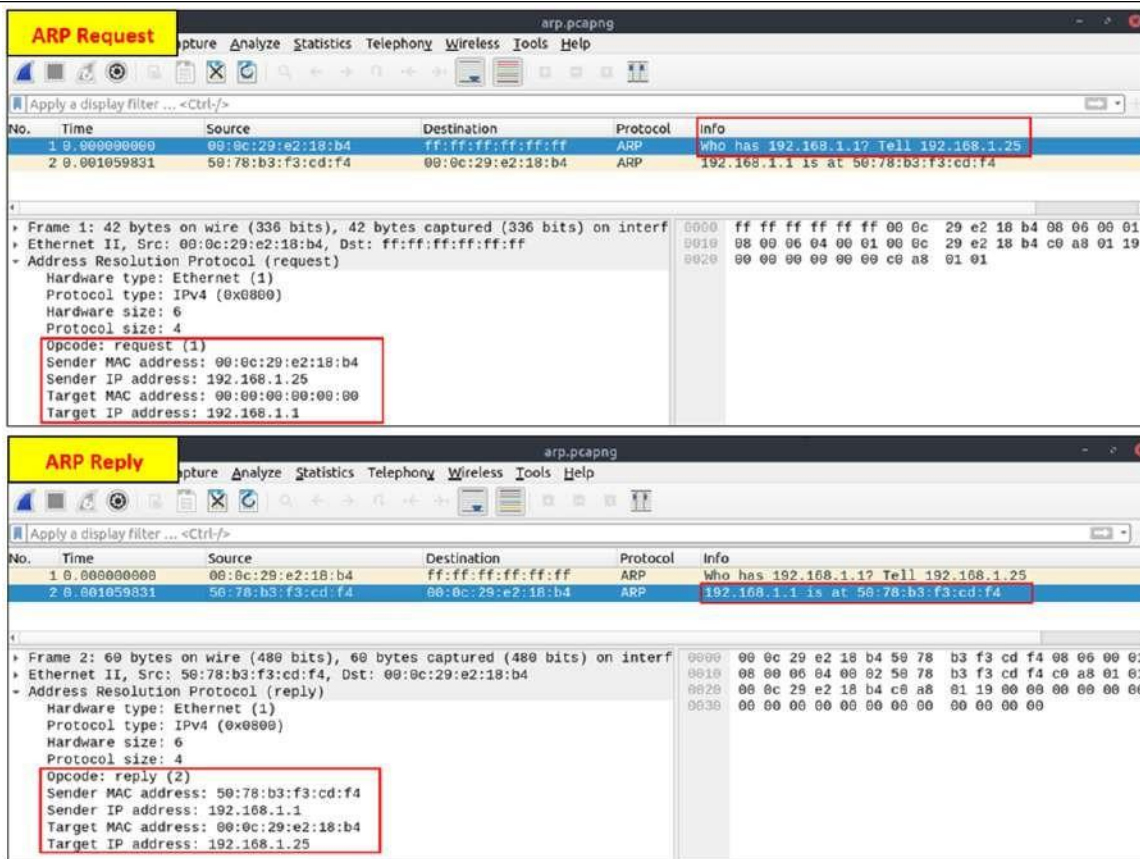
ARP analysis in a nutshell:

- Works on the local network
- Enables the communication between MAC addresses • Not a secure protocol
- Not a routable protocol
- It doesn't have an authentication function
- Common patterns are request & response, announcement and gratuitous packets.

Before investigating the traffic, let's review some legitimate and suspicious ARP packets. The legitimate requests are similar to the shown

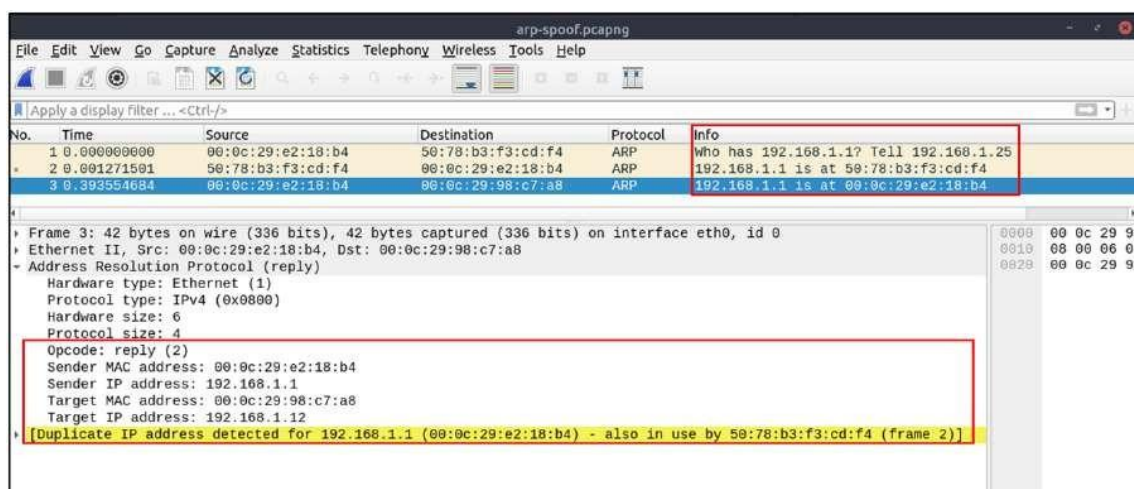
picture: a broadcast request that asks if any of the available hosts use an IP address and a reply from the host that uses the particular IP address.

Notes	Wireshark filter
Global search	• arp
"ARP" options for grabbing the low-hanging fruits: • Opcode 1: ARP requests. • Opcode 2: ARP responses. • Hunt: Arp scanning • Hunt: Possible ARP poisoning detection • Hunt: Possible ARP flooding from detection:	• arp.opcode == 1 • arp.opcode == 2 • arp.dst.hw_mac==00:00:00:00:00:00 • arp.duplicate- address-detected or arp.duplicate-address-frame • ((arp) && (arp.opcode == 1)) && (arp.src.hw_mac == target-mac-address)



A suspicious situation means having **two different ARP responses (conflict)** for a **particular IP address**.

In that case, Wireshark's expert info tab warns the analyst. However, it only shows the second occurrence of the duplicate value to highlight the conflict. Therefore, identifying the malicious packet from the legitimate one is the analyst's challenge. A possible IP spoofing case is shown in the picture below.



Here, knowing the network architecture and inspecting the traffic for a specific time frame can help detect the anomaly. As an analyst, you should take notes of your findings before going further. This will help you be organised and make it easier to correlate the further findings.

Look at the given picture; there is a conflict; the MAC address that ends with "b4" crafted an ARP request with the "192.168.1.25" IP address, then claimed to have the "192.168.1.1" IP address.

Notes	Detection Notes	Findings
Possible IP address match.	1 IP address announced from a MAC address.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.25
Possible ARP spoofing attempt.	2 MAC addresses claimed the same IP address (192.168.1.1). The "192.168.1.1" IP address is a possible gateway address.	• MAC1: 50:78:b3:f3:cd:f4 • MAC 2: 00:0c:29:e2:18:b4
Possible ARP flooding attempt.	The MAC address that ends with "b4" claims to have a different/new IP address.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.1

Let's keep inspecting the traffic to spot any other anomalies. Note that the case is split into multiple capture files to make the investigation easier.

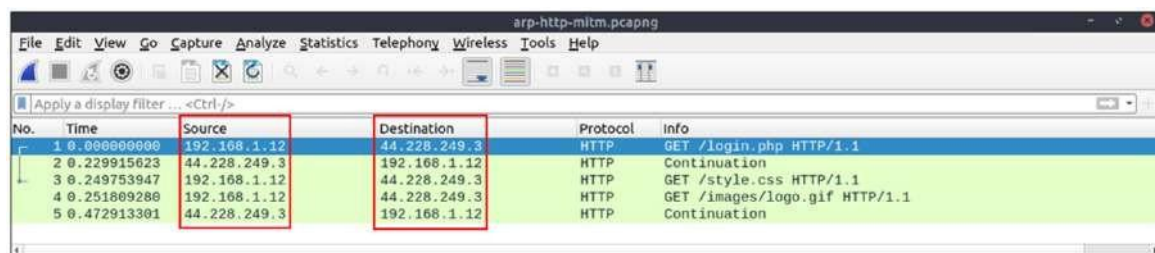
No.	Time	Source	Destination	Protocol	Info
1	0.000000000	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.1? Tell 192.168.1.25
2	0.001059831	50:78:b3:f3:cd:f4	00:0c:29:e2:18:b4	ARP	192.168.1.1 is at 50:78:b3:f3:cd:f4
3	0.010490253	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.37? Tell 192.168.1.25
4	0.020876839	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.158? Tell 192.168.1.25
5	0.031275021	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.212? Tell 192.168.1.25
6	0.041848453	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.176? Tell 192.168.1.25
7	0.052746298	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.73? Tell 192.168.1.25
8	0.063388601	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.216? Tell 192.168.1.25
9	0.073905794	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.181? Tell 192.168.1.25
10	0.084401792	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.217? Tell 192.168.1.25
11	0.095003040	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.173? Tell 192.168.1.25
12	0.105417559	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.136? Tell 192.168.1.25
13	0.115638938	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.132? Tell 192.168.1.25
14	0.125920898	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.130? Tell 192.168.1.25
15	0.136706415	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.254? Tell 192.168.1.25
16	0.147294383	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.232? Tell 192.168.1.25
17	0.157926474	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.162? Tell 192.168.1.25
18	0.168416859	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.109? Tell 192.168.1.25
19	0.178936116	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.253? Tell 192.168.1.25
20	0.189453050	00:0c:29:e2:18:b4	ff:ff:ff:ff:ff:ff	ARP	Who has 192.168.1.169? Tell 192.168.1.25

At this point, it is evident that there is an anomaly. A security analyst cannot ignore a flood of ARP requests. This could be malicious activity, scan or network problems. There is a new anomaly; the MAC address that ends with "b4" crafted multiple ARP requests with the "192.168.1.25" IP address. Let's focus on the source of this anomaly and extend the taken notes.

Notes	Detection Notes	Findings
Possible IP address match.	1 IP address announced from a MAC address.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.25
Possible ARP spoofing attempt.	2 MAC addresses claimed the same IP address (192.168.1.1).The " 192.168.1.1" IP address is a possible gateway address.	• MAC1: 50:78:b3:f3:cd:f4 • MAC 2: 00:0c:29:e2:18:b4
Possible ARP spoofing attempt.	The MAC address that ends with "b4" claims to have a different/new IP address.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.1
Possible ARP flooding attempt.	The MAC address that ends with "b4" crafted multiple ARP requests against a range of IP addresses.	• MAC: 00:0c:29:e2:18:b4 • IP: 192.168.1.xxx

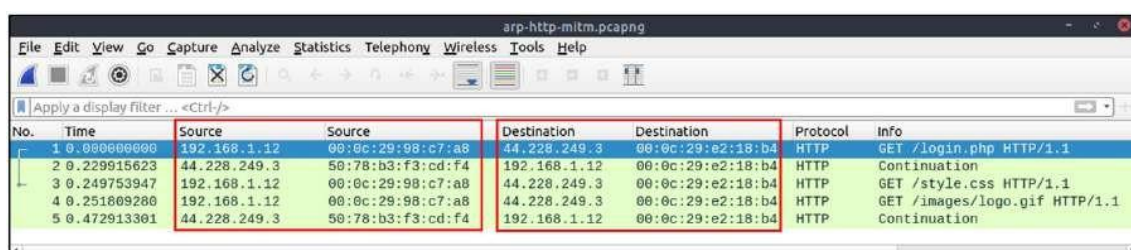
Up to this point, it is evident that the MAC address that ends with "b4" owns the "192.168.1.25" IP address and crafted suspicious ARP requests against a range of IP addresses. It also claimed to have the possible gateway address as well.

Let's focus on other protocols and spot the reflection of this anomaly in the following sections of the time frame.



No.	Time	Source	Destination	Protocol	Info
1	0.000000000	192.168.1.12	44.228.249.3	HTTP	GET /login.php HTTP/1.1
2	0.229915623	44.228.249.3	192.168.1.12	HTTP	Continuation
3	0.249753947	192.168.1.12	44.228.249.3	HTTP	GET /style.css HTTP/1.1
4	0.251809280	192.168.1.12	44.228.249.3	HTTP	GET /images/logo.gif HTTP/1.1
5	0.472913301	44.228.249.3	192.168.1.12	HTTP	Continuation

There is HTTP traffic, and everything looks normal at the IP level, so there is no linked information with our previous findings. Let's add the MAC addresses as columns in the packet list pane to reveal the communication behind the IP addresses.



No.	Time	Source	Source	Destination	Destination	Protocol	Info
1	0.000000000	192.168.1.12	00:0c:29:e2:18:b4	44.228.249.3	00:0c:29:e2:18:b4	HTTP	GET /login.php HTTP/1.1
2	0.229915623	44.228.249.3	50:78:b3:f3:cd:f4	192.168.1.12	00:0c:29:e2:18:b4	HTTP	Continuation
3	0.249753947	192.168.1.12	00:0c:29:e2:18:b4	44.228.249.3	00:0c:29:e2:18:b4	HTTP	GET /style.css HTTP/1.1
4	0.251809280	192.168.1.12	00:0c:29:e2:18:b4	44.228.249.3	00:0c:29:e2:18:b4	HTTP	GET /images/logo.gif HTTP/1.1
5	0.472913301	44.228.249.3	50:78:b3:f3:cd:f4	192.168.1.12	00:0c:29:e2:18:b4	HTTP	Continuation

One more anomaly! The MAC address that ends with "b4" is the destination of all HTTP packets! It is evident that there is a MITM attack, and the attacker is the host with the MAC address that ends with "b4". All traffic linked to "192.168.1.12" IP addresses is forwarded to the malicious host. Let's summarise the findings before concluding the investigation.

Detection Notes	Findings
IP to MAC matches.	3 IP to MAC address matches.
Attacker	The attacker created noise with ARP packets.
Router/gateway	Gateway address.
Victim	The attacker sniffed all traffic of the victim.

Detecting these bits and pieces of information in a big capture file is challenging. However, in real-life cases, you will not have "tailored data" ready for investigation. Therefore you need to have the analyst mindset, knowledge and tool skills to filter and detect the anomalies.

Identifying Hosts: DHCP, NetBIOS and Kerberos

Identifying Hosts

- When investigating a compromise or malware infection activity, a security analyst should know how to identify the hosts on the network apart from IP to MAC address match.
- One of the best methods is identifying the hosts and users on the network to decide the investigation's starting point and **list the hosts and users associated with the malicious traffic/activity**.
- Usually, enterprise networks use a predefined pattern to name users and hosts.

While this makes knowing and following the inventory easier, it has good and bad sides. The good side is that it will be easy to identify a user or host by looking at the name. The bad side is that it will be easy to clone that pattern and live in the enterprise network for adversaries. There are multiple solutions to avoid these kinds of activities, but for a security analyst, it is still essential to have host and user identification skills.

Protocols that can be used in Host and User identification:

- Dynamic Host Configuration Protocol (**DHCP**) traffic

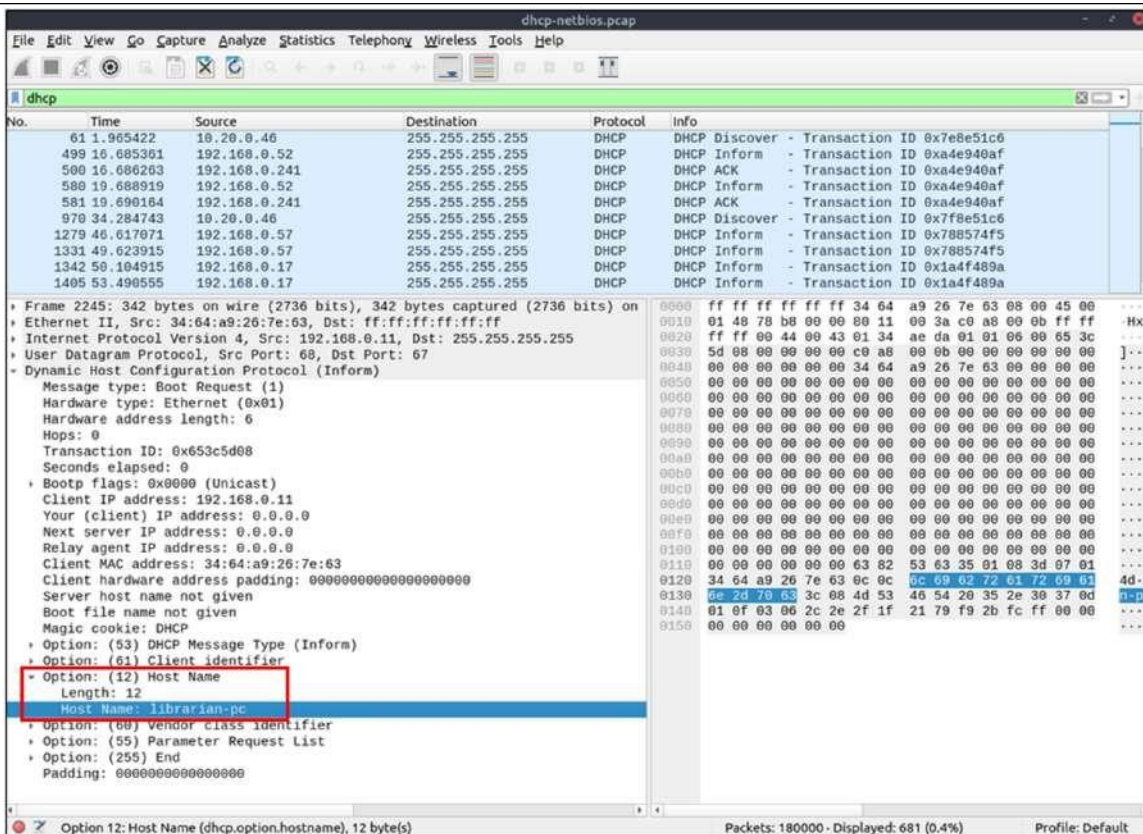
- **NetBIOS** (NBNS) traffic
- **Kerberos** traffic

DHCP

Dynamic Host Configuration Protocol (DHCP), is the technology responsible for managing automatic IP address and required communication parameters assignment.

DHCP investigation in a nutshell:

Notes	Wireshark Filter
Global search.	• <code>dhcp or bootp</code>
Filtering the proper DHCP packet options is vital to finding an event of interest. • "DHCP Request" packets contain the hostname information • "DHCP ACK" packets represent the accepted requests • "DHCP NAK" packets represent denied requests Due to the nature of the protocol, only "Option 53" (request type) has predefined static values. You should filter the packet type first, and then you can filter the rest of the options by "applying as column" or use the advanced filters like "contains" and "matches".	• Request: <code>dhcp.option.dhcp == 3</code> • ACK: <code>dhcp.option.dhcp == 5</code> • NAK: <code>dhcp.option.dhcp == 6</code>
"DHCP Request" options for grabbing the low-hanging fruits: • Option 12: Hostname. • Option 50: Requested IP address. • Option 51: Requested IP lease time. • Option 61: Client's MAC address.	• <code>dhcp.option.hostname contains "keyword"</code>
"DHCP ACK" options for grabbing the low-hanging fruits: • Option 15: Domain name. • Option 51: Assigned IP lease time.	• <code>dhcp.option.domain_name contains "keyword"</code>
"DHCP NAK" options for grabbing the low-hanging fruits: • Option 56: Message (rejection details/reason).	As the message could be unique according to the case/situation, It is suggested to read the message instead of filtering it. Thus, the analyst could create a more reliable hypothesis/result by understanding the event circumstances.

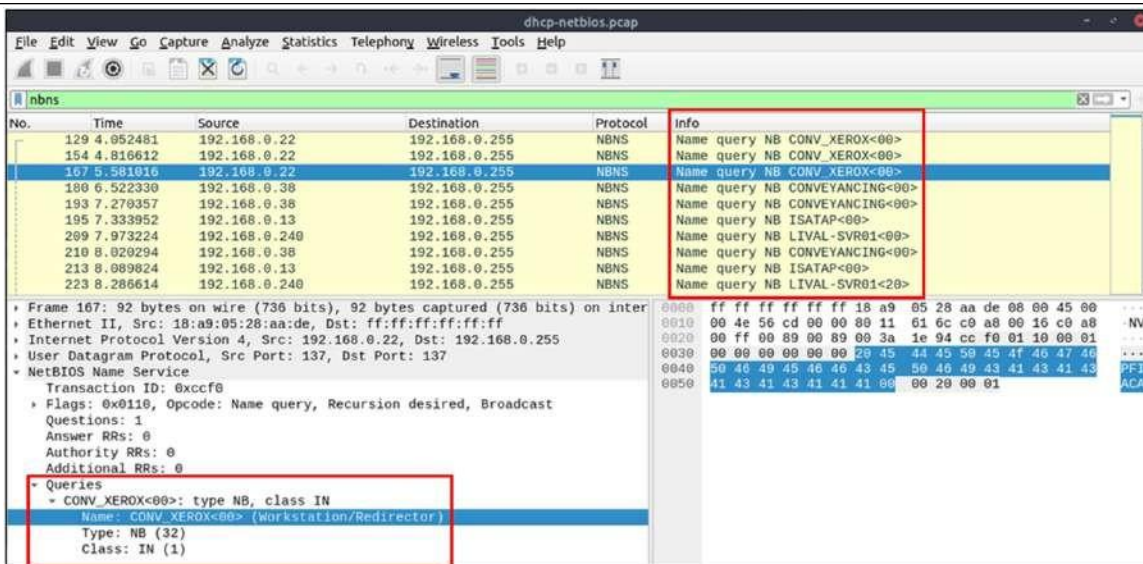


NetBIOS (NBNS) Analysis

NetBIOS or Network Basic Input/Output System is the technology responsible for allowing applications on different hosts to communicate with each other.

NBNS investigation in a nutshell:

Notes	Wireshark Filter
Global search.	• <code>nbns</code>
"NBNS" options for grabbing the low-hanging fruits: • Queries: Query details. • Query details could contain "name, Time to live (TTL) and IP address details"	• <code>nbns.name</code> contains "keyword"
NetBIOS registration requests	<code>nbns.flags.opcode == 5</code>

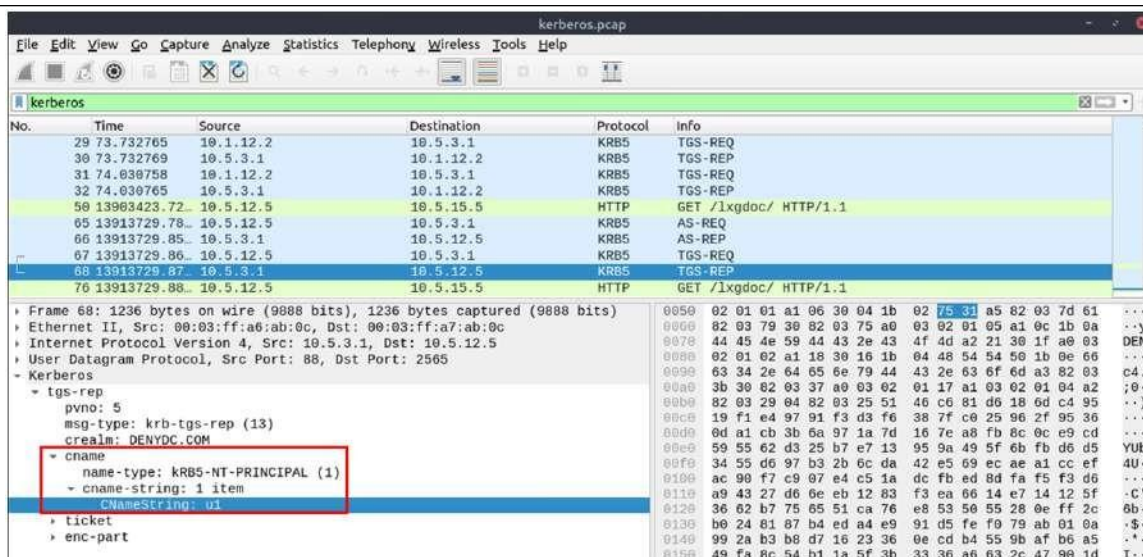


Kerberos Analysis

- **Kerberos** is the default authentication service for Microsoft Windows domains.
- It is responsible for authenticating service requests between two or more computers over the untrusted network. The ultimate aim is to **prove identity** securely.

Kerberos investigation in a nutshell:

Notes	Wireshark Filter
Global search.	• <code>kerberos</code>
User account search: • CNameString : The username . Note : Some packets could provide hostname information in this field. To avoid this confusion, filter the "\$" value. The values end with "\$" are hostnames, and the ones without it are user names.	• <code>kerberos.CNameString contains "keyword"</code> • <code>kerberos.CNameString and ! (kerberos.CNameString contains "\$")</code>
"Kerberos" options for grabbing the low-hanging fruits: • pvno : Protocol version. • realm : Domain name for the generated ticket. • sname : Service and domain name for the generated ticket. • addresses : Client IP address and NetBIOS name. Note : the "addresses" information is only available in request packets.	• <code>kerberos.pvno == 5</code> • <code>kerberos.realm contains ".org"</code> • <code>kerberos.SNameString == "krbtg"</code>



Tunneling Traffic: DNS and ICMP

Tunnelling Traffic: ICMP and DNS

- Traffic tunnelling is (also known as "**port forwarding**") transferring the data/resources in a secure method to network segments and zones.
- It can be used for "**internet to private networks**" and "**private networks to internet**" flow/direction.
- There is an **encapsulation process** to **hide** the data, so the transferred data appear natural for the case, but it contains private data packets and transfers them to the final destination securely.

Tunnelling provides **anonymity** and traffic security. Therefore it is highly used by enterprise networks. However, as it gives a significant level of data encryption, **attackers** use tunnelling to **bypass security perimeters** using the standard and trusted protocols used in everyday traffic like ICMP and DNS. Therefore, for a security analyst, it is crucial to have the ability to spot ICMP and DNS anomalies.

ICMP Analysis

- Internet Control Message Protocol (ICMP) is designed for diagnosing and reporting network communication issues. It is highly used in error reporting and testing.
- As it is a trusted network layer protocol, sometimes it is used for denial of service (DoS) attacks; also, adversaries use it in **data exfiltration and C2**

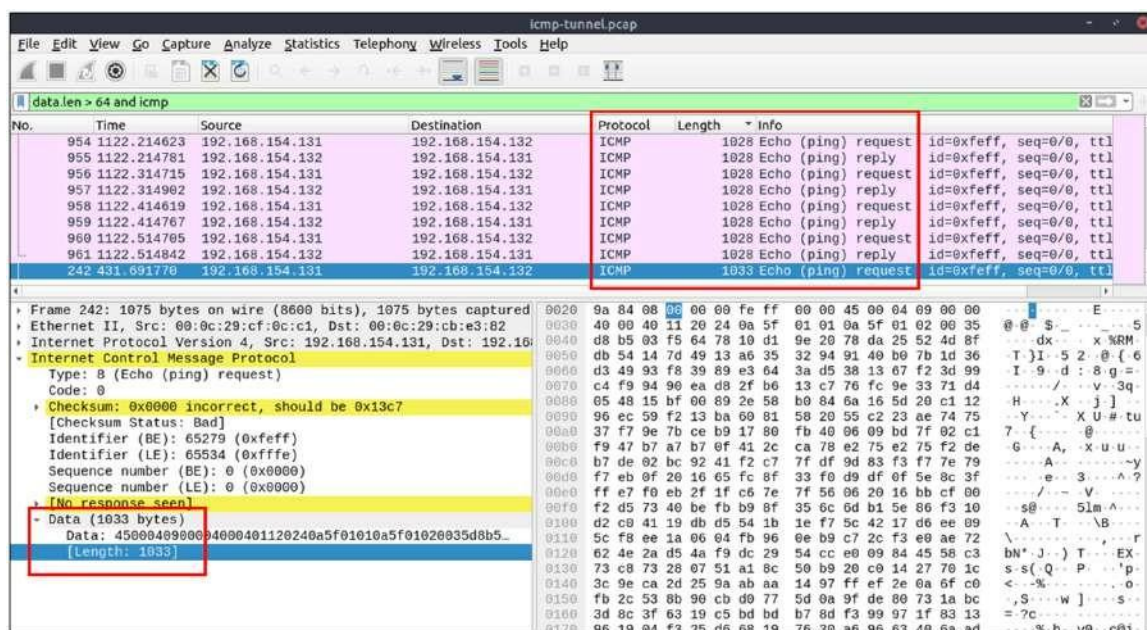
tunnelling activities.

ICMP analysis in a nutshell:

- Usually, ICMP tunnelling attacks are **anomalies** appearing/starting after a malware execution or vulnerability exploitation.
- As the ICMP packets can transfer an additional data payload, adversaries use this section to **exfiltrate data** and establish a C2 connection.
- It could be a TCP, HTTP or SSH data. As the ICMP protocols provide a great opportunity to carry extra data, it also has **disadvantages**.
 - Most enterprise networks **block** custom packets or require administrator privileges to create custom ICMP packets.

A large volume of ICMP traffic or anomalous packet sizes are indicators of ICMP tunnelling. Still, the adversaries could create custom packets that match the regular ICMP packet size (64 bytes), so it is still cumbersome to detect these tunnelling activities. However, a security analyst should know the normal and the abnormal to spot the possible anomaly and escalate it for further analysis.

Notes	Wireshark filters
Global search	icmp
"ICMP" options for grabbing the low-hanging fruits: • Packet length . • ICMP destination addresses . • Encapsulated protocol signs in ICMP payload.	data.len > 64 and icmp



DNS Analysis

- Domain Name System (DNS) is designed to **translate/convert IP domain addresses to IP addresses**.
- It is also known as a phonebook of the internet. As it is the essential part of web services, it is commonly used and trusted, and therefore often ignored. Due to that, **adversaries use it in data exfiltration and C2 activities**.

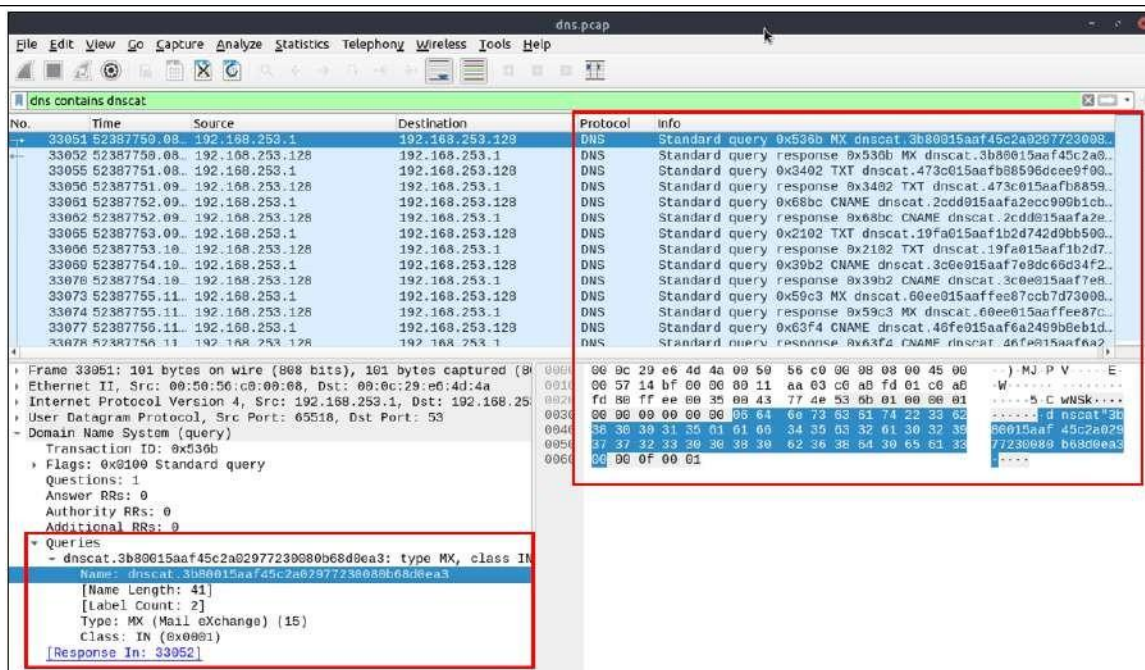
DNS analysis in a nutshell:

- Similar to ICMP tunnels, DNS attacks are anomalies appearing/starting after a malware execution or vulnerability exploitation.
- Adversary creates (or already has) a **domain address** and configures it as a **C2 channel**. The malware or the commands executed after exploitation sends **DNS queries to the C2 server**.
- However, these queries are **longer than default DNS queries** and crafted for subdomain addresses. Unfortunately, these subdomain addresses are not actual addresses; they are encoded commands as shown below:

"encoded-commands.maliciousdomain.com"

- When this query is routed to the C2 server, the server sends the actual malicious commands to the host.
- As the DNS queries are a natural part of the networking activity, these packets have the chance of not being detected by network perimeters. A security analyst should know how to investigate the DNS packet lengths and target addresses to spot these anomalies.

Notes	Wireshark Filter
Global search	• dns
"DNS" options for grabbing the low-hanging fruits: • Query length . • Anomalous and non-regular names in DNS addresses. • Long DNS addresses with encoded subdomain addresses . • Known patterns like dnscat and dns2tcp . • Statistical analysis like the anomalous volume of DNS requests for a particular target. !mdns: Disable local link device queries.	• dns contains "dnscat" • dns.qry.name.len > 15 and !mdns



Cleartext Protocol Analysis: FTP

Cleartext Protocol Analysis

Investigating cleartext protocol traces sounds easy, but when the time comes to investigate a big network trace for incident analysis and response, the game changes. Proper analysis is more than following the stream and reading the cleartext data. For a security analyst, it is important to create statistics and key results from the investigation process. As mentioned earlier at the beginning of the Wireshark room series, the analyst should have the required network knowledge and tool skills to accomplish this. Let's simulate a cleartext protocol investigation with Wireshark!

FTP Analysis

File Transfer Protocol (FTP) is designed to transfer files with ease, so it focuses on simplicity rather than security. As a result of this, using this protocol in unsecured environments could create security issues like:

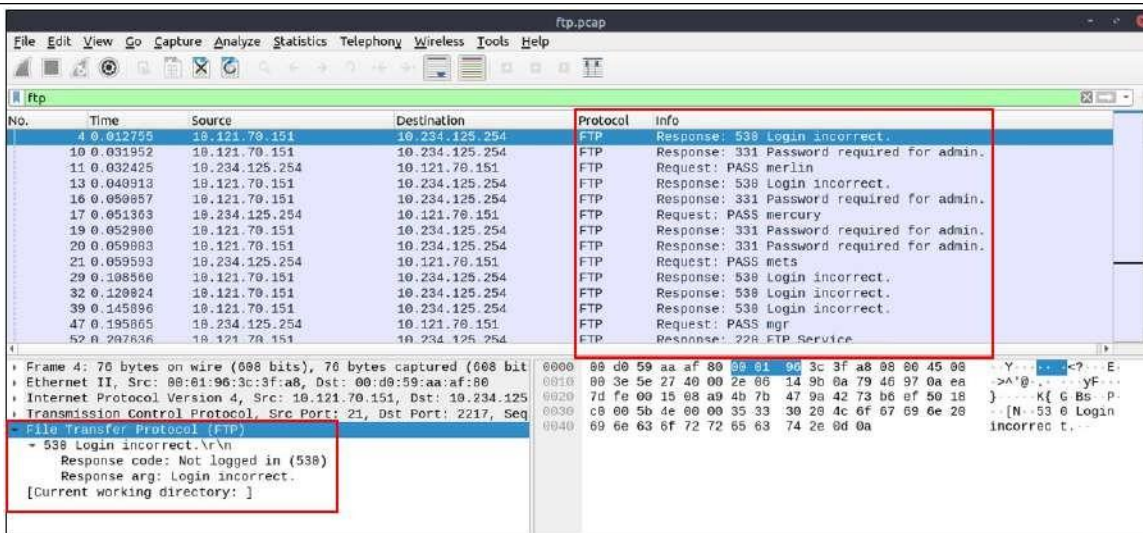
- MITM attacks
- Credential stealing and unauthorised access • Phishing

- Malware planting • Data

exfiltration

FTP analysis in a nutshell:

Notes	Wireshark Filter
Global search	• <code>ftp</code>
"FTP" options for grabbing the low-hanging fruits: • x1x series: Information request responses. • x2x series: Connection messages. • x3x series: Authentication messages. Note: "200" means command successful .	---
"x1x" series options for grabbing the low-hanging fruits: • 211: System status. • 212: Directory status. • 213: File status	• <code>ftp.response.code == 211</code>
"x2x" series options for grabbing the low-hanging fruits: • 220: Service ready. • 227: Entering passive mode. • 228: Long passive mode. • 229: Extended passive mode.	• <code>ftp.response.code == 227</code>
"x3x" series options for grabbing the low-hanging fruits: • 230: User login. • 231: User logout. • 331: Valid username. • 430: Invalid username or password • 530: No login, invalid password.	• <code>ftp.response.code == 230</code>
"FTP" commands for grabbing the low-hanging fruits: • USER: Username. • PASS: Password. • CWD: Current work directory. • LIST: List.	• <code>ftp.request.command == "USER"</code> • <code>ftp.request.command == "PASS"</code> • <code>ftp.request.arg == "password"</code>
Advanced usages examples for grabbing low- hanging fruits: • Bruteforce signal: List failed login attempts. • Bruteforce signal: List target username. • Password spray signal: List targets for a static password.	• <code>ftp.response.code == 530</code> • <code>(ftp.response.code == 530) and (ftp.response.arg contains "username")</code> • <code>(ftp.request.command == "PASS") and (ftp.request.arg == "password")</code>



Cleartext Protocol Analysis: HTTP

HTTP Analysis

- Hypertext Transfer Protocol (HTTP) is a cleartext-based, request-response and client-server protocol. It is the standard type of network activity to request/serve web pages, and by default, it is not blocked by any network perimeter. As a result of being unencrypted and the backbone of web traffic, HTTP is one of the must-to-know protocols in traffic analysis. Following attacks could be detected with the help of HTTP analysis:

- Phishing pages
- Web attacks
- Data exfiltration
- Command and control traffic (C2) HTTP analysis in a

nutshell:

Notes	Wireshark Filter
Global search Note: HTTP2 is a revision of the HTTP protocol for better performance and security. It supports binary data transfer and request&response multiplexing.	- http - http2
"HTTP Request Methods" for grabbing the low-hanging fruits: • GET • POST • Request: Listing all requests	- http.request.method == "GET" - http.request.method == "POST" - http.request
"HTTP Response Status Codes" for grabbing the low-hanging	http.response.code ==

fruits: • 200 OK: Request successful. • 301 Moved Permanently: Resource is moved to a new URL/path (permanently). • 302 Moved Temporarily: Resource is moved to a new URL/path (temporarily). • 400 Bad Request: Server didn't understand the request. • 401 Unauthorised: URL needs authorisation (login, etc.). • 403 Forbidden: No access to the requested URL. • 404 Not Found: Server can't find the requested URL. • 405 Method Not Allowed: Used method is not suitable or blocked. • 408 Request Timeout: Request look longer than server wait time. • 500 Internal Server Error: Request not completed, unexpected error. • 503 Service Unavailable: Request not completed server or service is down.	<pre>200 • http.response.code == 401 • http.response.code == 403 • http.response.code == 404 • http.response.code == 405 • http.response.code == 503</pre>
"HTTP Parameters" for grabbing the low-hanging fruits: • User agent: Browser and operating system identification to a web server application. • Request URI: Points the requested resource from the server. • Full *URI: Complete URI information. *URI: Uniform Resource Identifier.	<pre>• http.user_agent contains "nmap" • http.request.uri contains "admin" • http.request.full_uri contains "admin"</pre>
"HTTP Parameters" for grabbing the low-hanging fruits: • Server: Server service name. • Host: Hostname of the server • Connection: Connection status. • Line-based text data: Cleartext data provided by the server. • HTML Form URL Encoded: Web form information.	<pre>• http.server contains "apache" • http.host contains "keyword" • http.host == "keyword" • http.connection == "Keep-Alive" • data-text- lines contains "keyword"</pre>

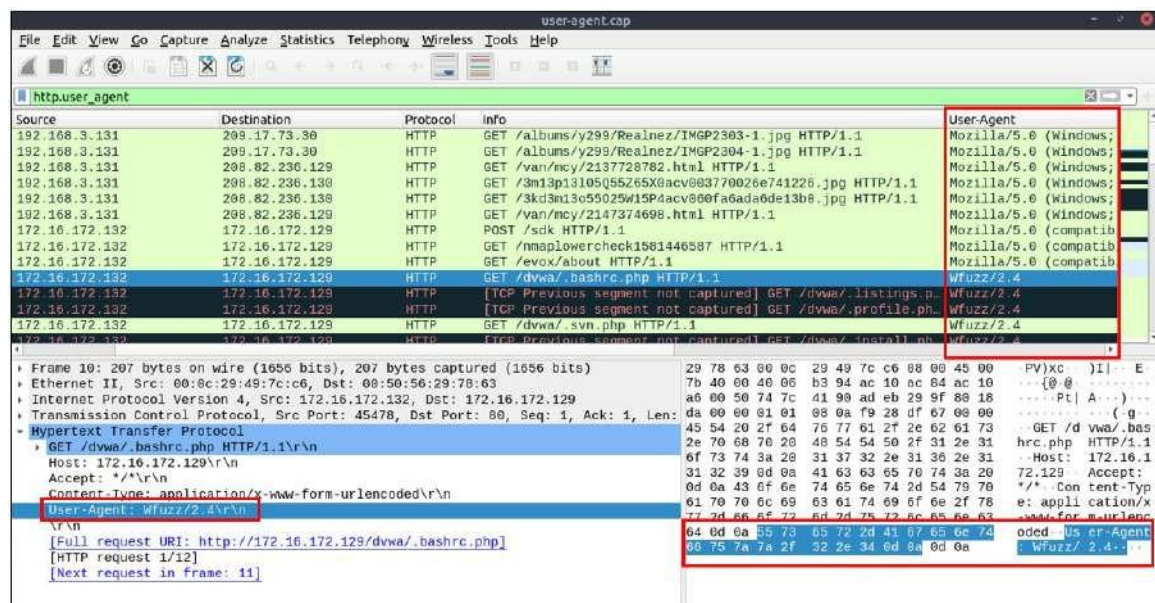
User Agent Analysis

- As the adversaries use sophisticated technics to accomplish attacks, they try to leave traces similar to natural traffic through the known and trusted protocols.
- For a security analyst, it is important to spot the anomaly signs on the bits and pieces of the packets. The "user-agent" field is one of the great resources for spotting anomalies in HTTP traffic.
- In some cases, adversaries successfully **modify** the user-agent data, which could look super natural.
- A security analyst cannot rely only on the user-agent field to spot an anomaly.
- **Never whitelist a user agent**, even if it looks natural.

- User agent-based anomaly/threat detection/hunting is an additional data source to check and is useful when there is an obvious anomaly. If you are unsure about a value, you can conduct a web search to validate your findings with the default and normal user-agent info ([example site](#)).

User Agent analysis in a nutshell:

Notes	Wireshark Filter
Global search.	• <code>http.user_agent</code>
Research outcomes for grabbing the low-hanging fruits: • Different user agent information from the same host in a short time notice. • Non-standard and custom user agent info. • Subtle spelling differences. ("Mozilla" is not the same as "Mozlilla" or "Mozlila") • Audit tools info like Nmap, Nikto, Wfuzz and sqlmap in the user agent field. • Payload data in the user agent field.	• <code>(http.user_agent contains "sqlmap") or (http.user_agent contains "Nmap") or (http.user_agent contains "Wfuzz") or (http.user_agent contains "Nikto")</code>



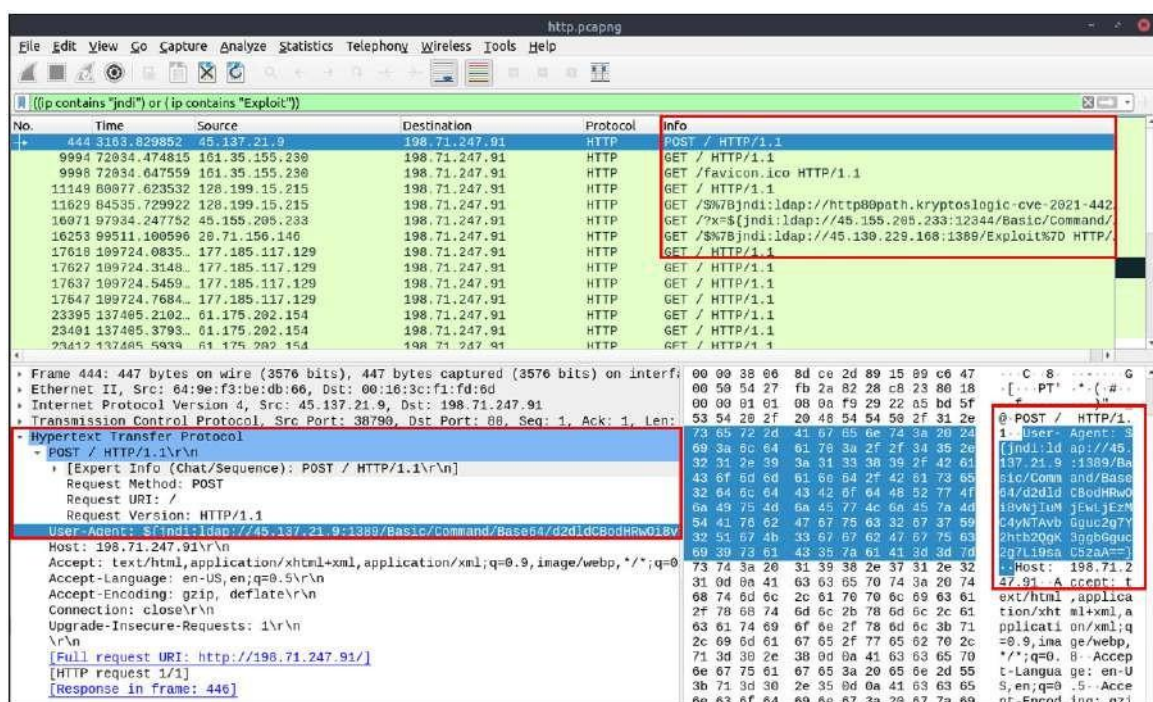
Log4j Analysis

- Log4j Analysis refers to the process of examining and assessing the logs generated by the Apache Log4j framework in order to identify security vulnerabilities, operational issues, or other relevant insights within an application or system.

- A proper investigation starts with prior research on threats and anomalies going to be hunted. Let's review the knowns on the "Log4j" attack before launching Wireshark.

Log4j vulnerability analysis in a nutshell:

Notes	Wireshark Filters
Research outcomes for grabbing the low-hanging fruits: • The attack starts with a "POST" request • There are known cleartext patterns: "jndi:ldap" and "Exploit.class".	<pre>http.request.method == "POST" • (ip contains "jndi") or (ip contains "Exploit") • (frame contains "jndi") or (frame contains "Exploit") • (http.user_agent contains "\$") or (http.user_agent contains "==")</pre>



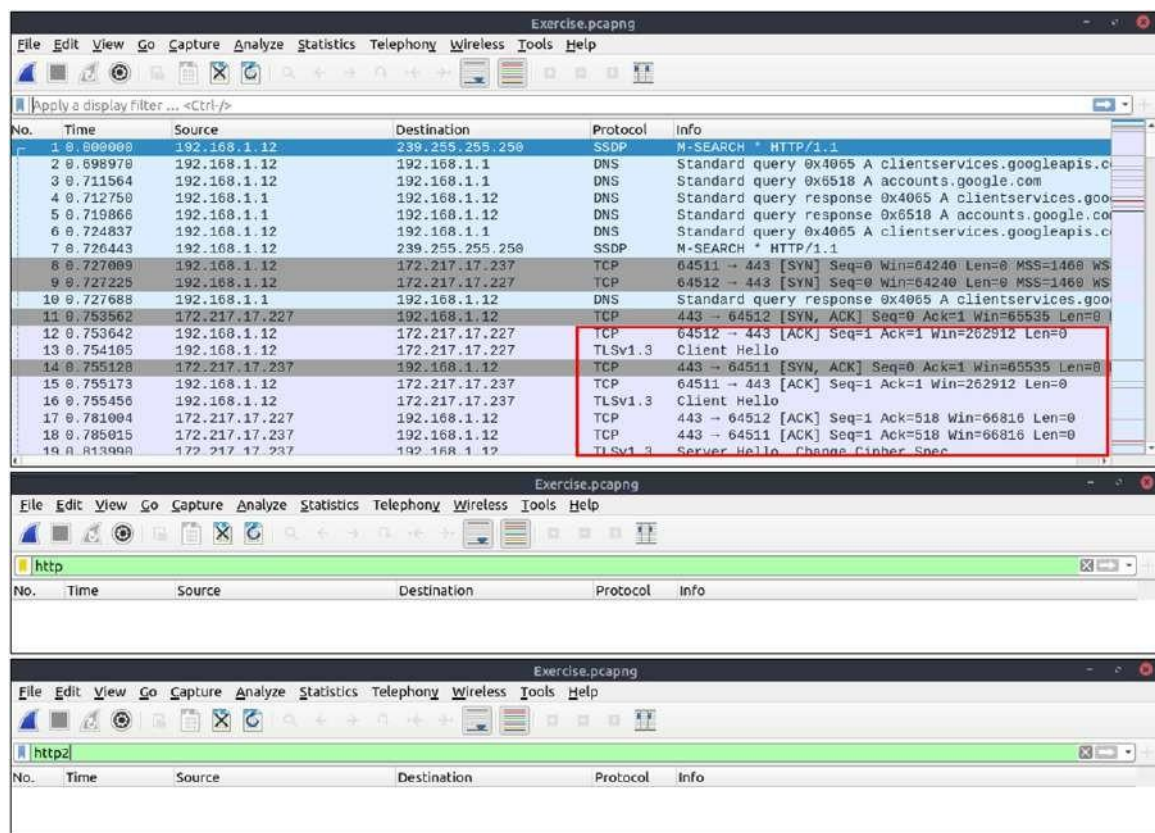
Encrypted Protocol Analysis: Decrypting HTTPS

Decrypting HTTPS Traffic

- When investigating web traffic, analysts often run across encrypted traffic. This is caused by using the Hypertext Transfer Protocol Secure (HTTPS) protocol for enhanced security **against spoofing, sniffing and intercepting attacks**.

- HTTPS uses **TLS** protocol to **encrypt** communications, so it is impossible to decrypt the traffic and view the transferred data without having the **encryption/decryption key pairs**.
- As this protocol provides a good level of security for transmitting sensitive data, attackers and malicious websites also use HTTPS.
- Therefore, a security analyst should know how to use key files to decrypt encrypted traffic and investigate the traffic activity.

The packets will appear in different colours as the HTTP traffic is encrypted. Also, protocol and info details (actual URL address and data returned from the server) will not be fully visible. The first image below shows the HTTP packets encrypted with the TLS protocol. The second and third images demonstrate filtering HTTP packets without using a key log file.



Additional information for HTTPS :

Notes	Wireshark Filter
"HTTPS Parameters" for grabbing the low-hanging fruits: • Request: Listing all requests • TLS: Global TLS search • TLS Client Request • TLS Server response • Local Simple Service Discovery Protocol (SSDP) Note: SSDP is a network protocol that provides advertisement and discovery of network services.	• http.request • tls • tls.handshake.type = 1 • • tls.handshake.type = 2 • ssdp

Similar to the TCP three-way handshake process, the TLS protocol has its handshake process.

The first two steps contain "**Client Hello**" and "**Server Hello**" messages.

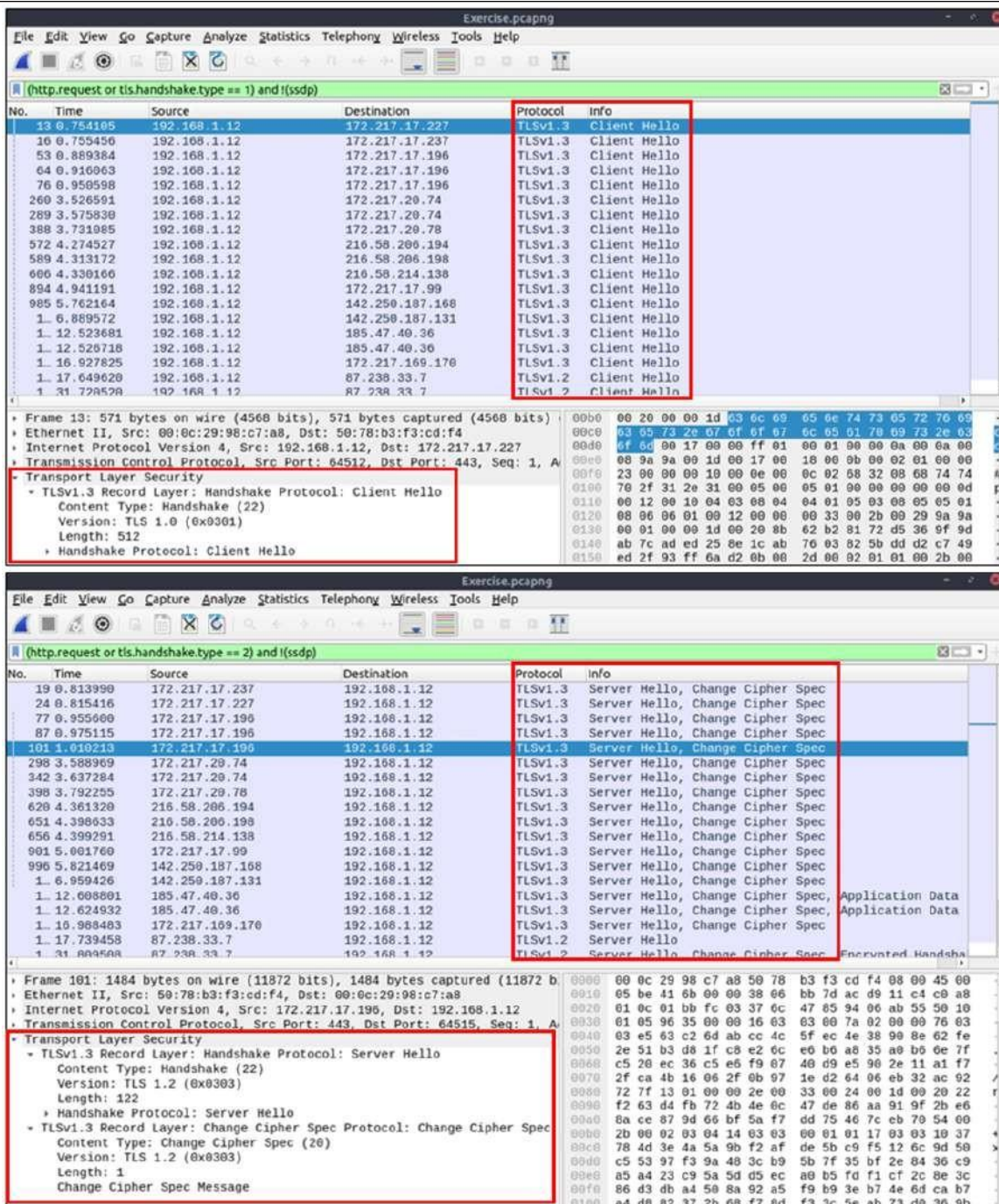
The given filters show the initial hello packets in a capture file. These filters are helpful to spot which IP addresses are involved in the TLS handshake.

Client Hello:

```
(http.request or tls.handshake.type == 1) and !(ssdp)
```

Server Hello:

```
(http.request or tls.handshake.type == 2) and !(ssdp)
```



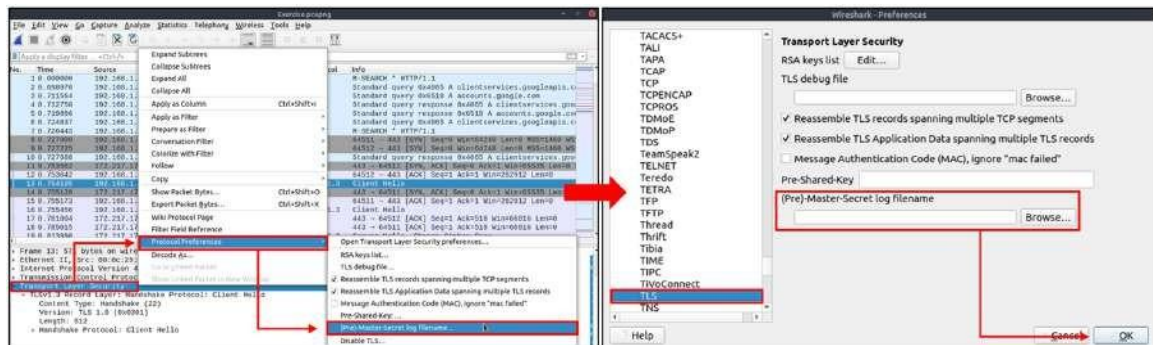
An **encryption key log file** is a text file that contains unique key pairs to decrypt the encrypted traffic session.

These key pairs are automatically created (**per session**) when a connection is established with an SSL/TLS-enabled webpage.

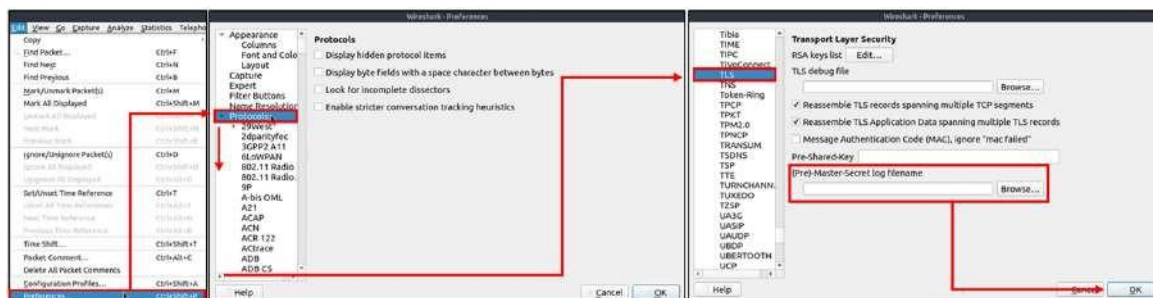
As these processes are all accomplished in the browser, you need to configure your system and use a suitable browser (Chrome and Firefox support this) to save these values as a key log file.

- To do this, you will need to set up an environment variable and create the **SSLKEYLOGFILE**, and the browser will dump the keys to this file as you browse the web.
- SSL/TLS key pairs are created per session at the connection time, so it is important to dump the keys during the traffic capture. Otherwise, it is not possible to create/generate a suitable key log file to decrypt captured traffic.
- You can use the "right-click" menu or "**Edit --> Preferences --> Protocols --> TLS**" menu to add/remove key log files.

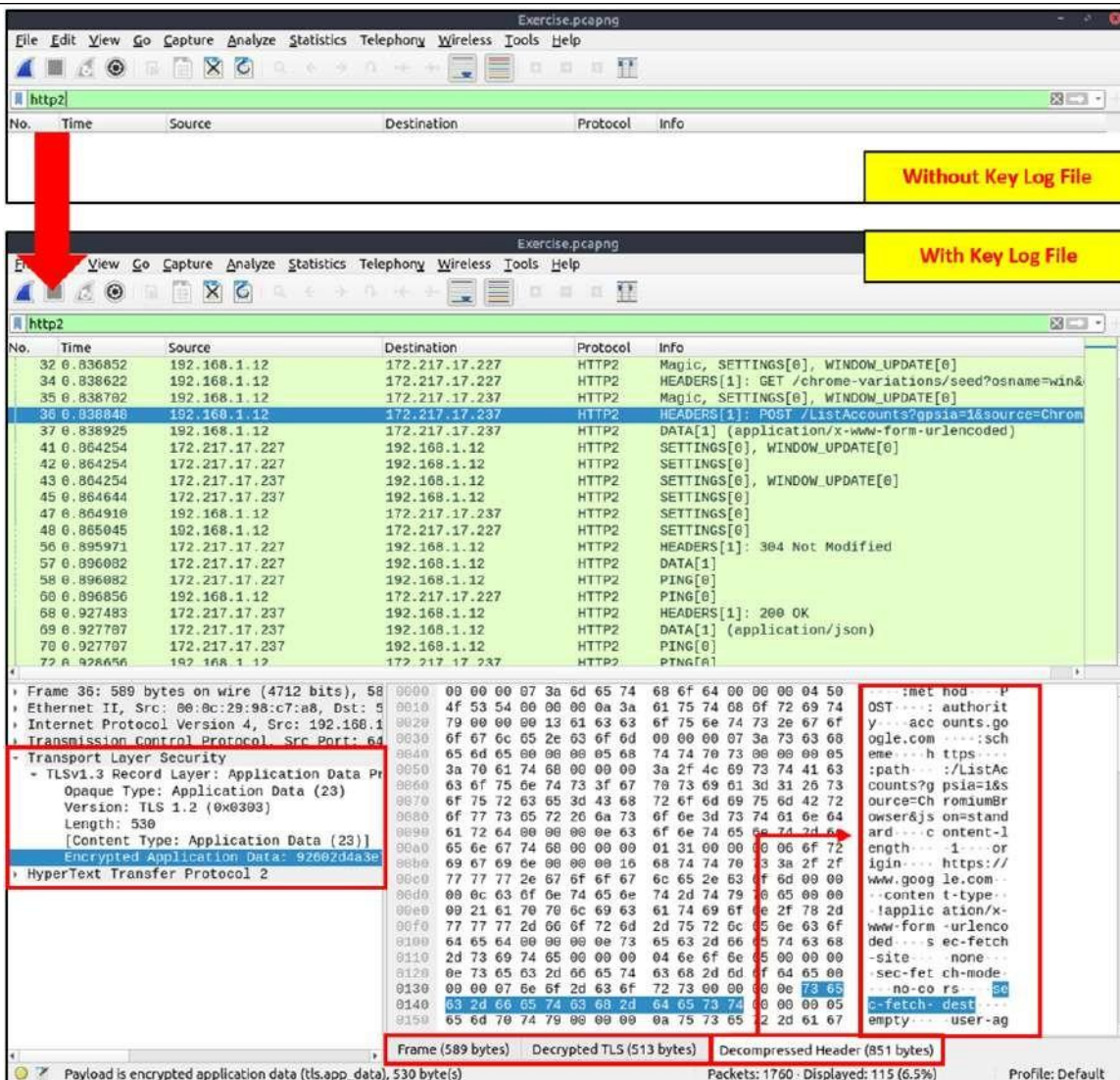
Adding key log files with the "right-click" menu:



Adding key log files with the "**Edit --> Preferences --> Protocols --> TLS**" menu:



Viewing the traffic with/without the key log files:



The above image shows that the traffic details are visible after using the key log file. Note that the packet details and bytes pane provides the data in different formats for investigation. Decompressed header info and HTTP2 packet details are available after decrypting the traffic. Depending on the packet details, you can also have the following data formats:

- Frame
- Decrypted TLS
- Decompressed Header
- Reassembled

TCP

- Reassembled SSL

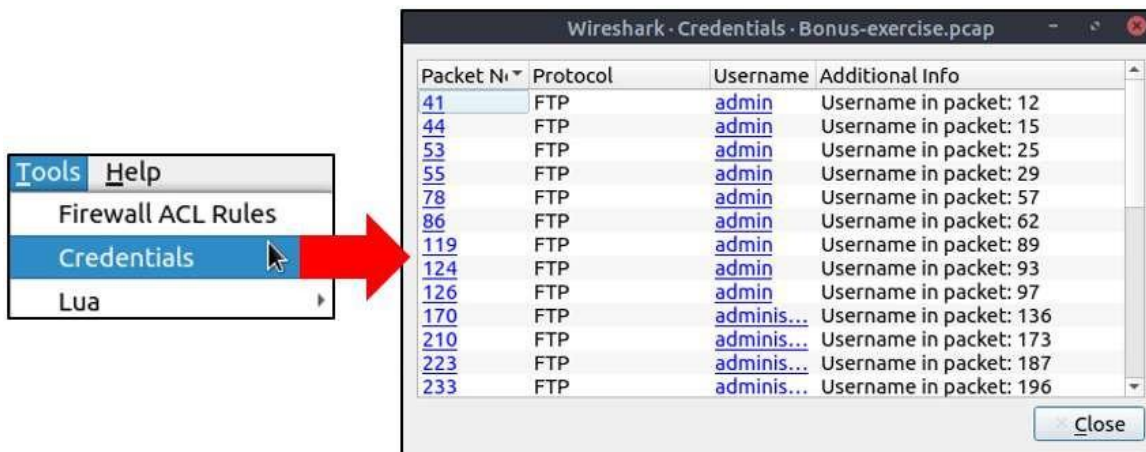
Hunt Cleartext Credentials!

Bonus: Hunt Cleartext Credentials!

Some Wireshark dissectors (FTP, HTTP, IMAP, pop and SMTP) are programmed to extract cleartext passwords from the capture file. You can view detected credentials using the "**Tools --> Credentials**" menu. This

feature works only after specific versions of Wireshark (v3.1 and later). Since the feature works only with particular protocols, it is suggested to have manual checks and not entirely rely on this feature to decide if there is a cleartext credential in the traffic.

Once you use the feature, it will open a new window and provide detected credentials. It will show the packet number, protocol, username and additional information. This window is clickable; clicking on the packet number will select the packet containing the password, and clicking on the username will select the packet containing the username info. The additional part prompts the packet number that contains the username.



Actionable Results!

- Wireshark is not all about packet details; it can help you to create **firewall rules** ready to implement with a couple of clicks.
- You can create firewall rules by using the "**Tools --> Firewall ACL Rules**" menu. Once you use this feature, it will open a new window and provide a combination of rules (IP, port and MAC address-based) for different purposes. Note that these rules are generated for implementation on an outside firewall interface.

Currently, Wireshark can create rules for:

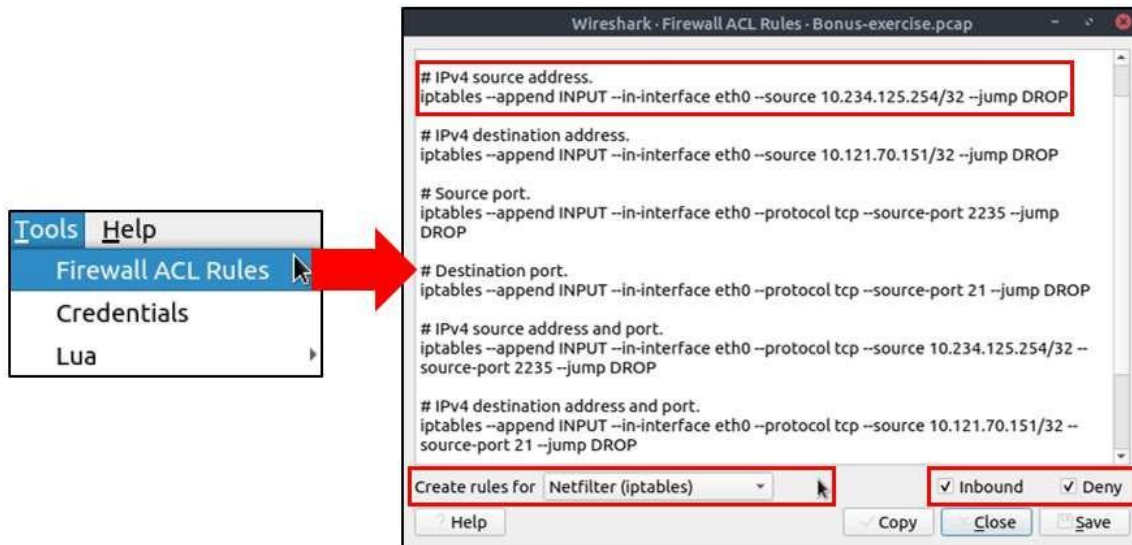
• Netfilter (iptables)

- Cisco IOS (standard/extended)

IP Filter (ipfilter) IPFirewall

(ipfw) Packet filter (pf)

Windows Firewall (netsh new/old format)



Tasks:

Task 1: Capturing and Exporting Objects

1. Capture network traffic using Wireshark.
2. Identify and extract transferred files from HTTP, SMB, or TFTP streams.
3. Document findings with screenshots.

Task 2: Time Display Format Analysis

1. Change Wireshark's time display format to UTC.
2. Compare and analyze packet timing differences.
3. Submit observations with relevant screenshots.

Task 3: Expert Info Analysis

1. Identify anomalies using Wireshark's Expert Info feature.
2. Classify errors, warnings, and malformed packets.
3. Provide a detailed explanation of identified issues.

Task 4: Applying Filters for Traffic Analysis

1. Use "Apply as Filter" to isolate specific network traffic.
2. Utilize conversation filters to analyze related packets.
3. Provide examples and screenshots of applied filters.

Task 5: Analyzing HTTP Streams

1. Follow an HTTP stream to reconstruct application-layer data.
2. Identify and document potential sensitive information.
3. Submit a report including step-by-step findings.

Task 6: Statistical Analysis

1. Use the Statistics menu to analyze protocol hierarchy and endpoints.
2. Extract key insights about network traffic patterns.
3. Document results and interpretations.

Task 7: Resolving Addresses and Protocol Hierarchy

1. Use Wireshark to resolve IP addresses to hostnames.
2. View the protocol hierarchy to analyze network traffic structure.
3. Submit a summary of key findings.

Submission Requirements:

- A detailed report including:
 - Screenshots of each task performed.
 - Explanation of results and insights gained.
 - Summary of findings and security recommendations.
- Submit the report by Next Lab.

This assignment will help students develop practical skills in **network packet analysis** using **Wireshark**. Good luck!